

云原生训练营

模块十一： 将应用迁移至 Kubernetes 平台

孟凡杰

前 eBay 资深架构师



目录

1. 应用接入最佳实践
 - 应用容器化
 - 将应用迁移至 Kubernetes
 - 有状态应用的复杂性讨论
2. Spec 管理神器 - Helm
3. metrics-server
4. 自动扩容缩容 - HPA
5. 自动扩容缩容 - VPA
6. 如何解决社区基础弹性能力不足的问题

1. 应用接入最佳实践

应用容器化

目标

稳定性、可用性、性能、安全

从多维度思考高可用的问题

单个实例视角

资源需求
配置管理
数据保存
日志和指标收集

应用视角

冗余部署
部署多少个实例
负载均衡
健康检查
服务发现
监控
故障转移
扩缩容

安全视角

镜像安全
应用安全
数据安全
通讯安全

应用容器化的思考

应用本身

- 启动速度
- 健康检查
- 启动参数

Dockerfile

- 用什么基础镜像
 - 基础镜像越小越好
- 需要装什么 Utility
 - lib 越少越好?
- 多少个进程
 - 主次要分清楚, 哪个是决定状态的主程序
 - Fork bomb 的危害
- 代码 (应用程序) 和配置分离
 - 配置如何管理
 - 环境变量
 - 配置文件
- 分层的控制
- Entrypoint

思考: GOMAXPROCS会如何设置

容器额外开销和风险

Log driver

- Blocking mode
- Non blocking mode

共用 kernel 所以

- 系统参数配置共享
- 进程数共享 - Fork bomb
- fd 数共享
- 主机磁盘共享

容器化应用的资源监控

容器中看到的资源是主机资源

- Top
- Java runtime.GetAvailableProcesses()
- cat /proc/cpuinfo
- cat /proc/meminfo
- df -k

解决方案

- 查询 /proc/1/cgroup 是否包含 kubepods 关键字（docker 关键字不可靠）。
 - 11:cpu,cpuacct:/kubepods.slice/kubepods-besteffort.slice/kubepods-besteffort-pod521722c3_85a8_11e9_87fc_3cfdfe57c998.slice/9568cc0d8ae182395e1ce172e2cac723c4781a999e89e0f9f10d33af079a56e9
- 包含此关键字，则表明是运行在 Kubernetes 之上。

内存开销

配额

- `cat /sys/fs/cgroup/memory/memory.limit_in_bytes`
- 36854771712

用量

- `cat /sys/fs/cgroup/memory/memory.usage_in_bytes`
- 448450560

CPU

配额, 分配的 CPU 个数 = $quota / period$, $quota = -1$ 代表 besteffort

- `cat /sys/fs/cgroup/cpu/cpu.cfs_quota_us`
 - -1
- `cat /sys/fs/cgroup/cpu/cpu.cfs_period_us`
 - 100000

用量

- `cat /sys/fs/cgroup/cpuacct/cpuacct.usage_percpu` (按CPU区分)
 - 140669504971 148500278385 149957919463 152786448674
- `cat /sys/fs/cgroup/cpuacct/cpuacct.usage`
 - 12081100465458

其他方案

lxcfs

- 通过 so 挂载的方式，使容器获得正确的资源信息

Kata

- VM 中跑 container

Virtlet

- 直接启动 VM

对应用造成的影响

Java

- Concurrent GC Thread;
- Heap Size;
- 线程数不可控。

Node.js

- 多线程模式启动的 Thread 数量过多，导致 OOM Kill。

将应用迁移至 Kubernetes

Pod spec

初始化需求 (init container)

需要几个主 container

权限? Privilege 和 SecurityContext(PSP)

共享哪些 Namespace(PID, IPC, NET, UTS, MNT)

配置管理

优雅终止

健康检查

- Liveness Probe
- Readiness Probe

DNS 策略以及对 resolv.conf 的影响

imagePullPolicy Image 拉取策略

Probe 误用会造成严重后果

readinessProbe:

exec:

command:

- /opt/rprobe.sh

failureThreshold: 3

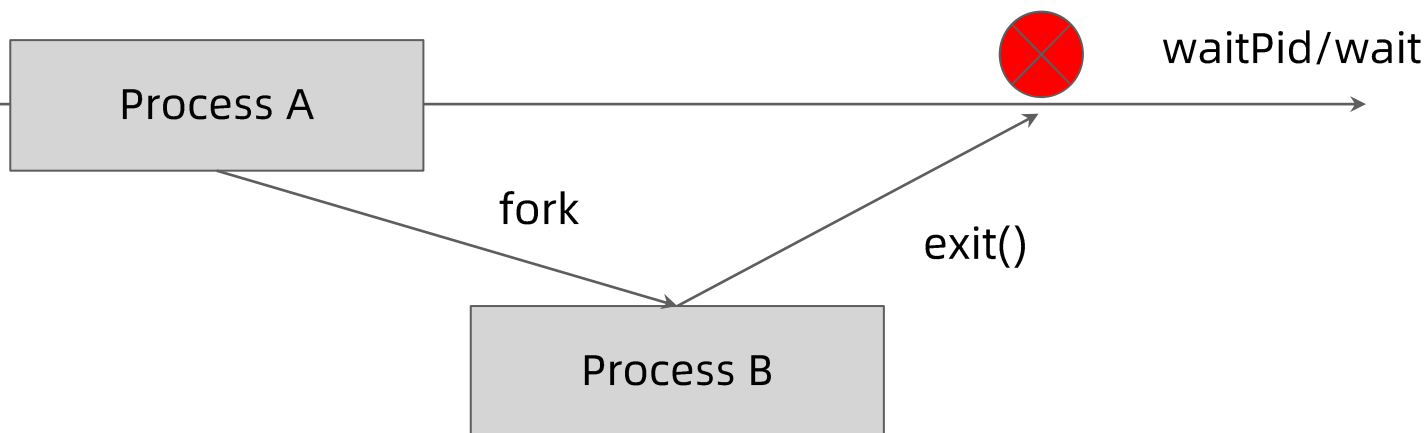
initialDelaySeconds: 30

periodSeconds: 10

successThreshold: 1

timeoutSeconds: 1

```
-bash-4.2# ps -efl | grep 47924
1 Z root      566 47924  0  80  0 -    0 -    Jul09 ?      00:00:00 [rprobe.sh] <defunct>
0 Z root      884 47924  0  80  0 -    0 -    Jun23 ?      00:00:00 [tee] <defunct>
0 Z root     1221 47924  0  80  0 -    0 -    May05 ?      00:00:00 [tee] <defunct>
```



后果：可能耗尽节点所有的PID

如何防止 PID 泄露

单进程容器

合理的处理多进程容器

- 容器的初始化进程必须负责清理 fork 出来的所有子进程
- 开源方案
 - Tini <https://github.com/krallin/tini>
 - 采用 Tini 作为容器的初始化进程 (PID=1) , 容器中僵尸进程的父进程会被置为1

如果不采用特殊初始化进程

- 建议采用 HTTPCheck 作为 Probe
- 为 exec Probe 设置合理的超时时间

```
ENTRYPOINT ["/tini", "--"]
```

```
# Run your program under Tini  
CMD ["/your/program", "-and", "-its", "arguments"]  
# or docker run your-image /your/program ...
```

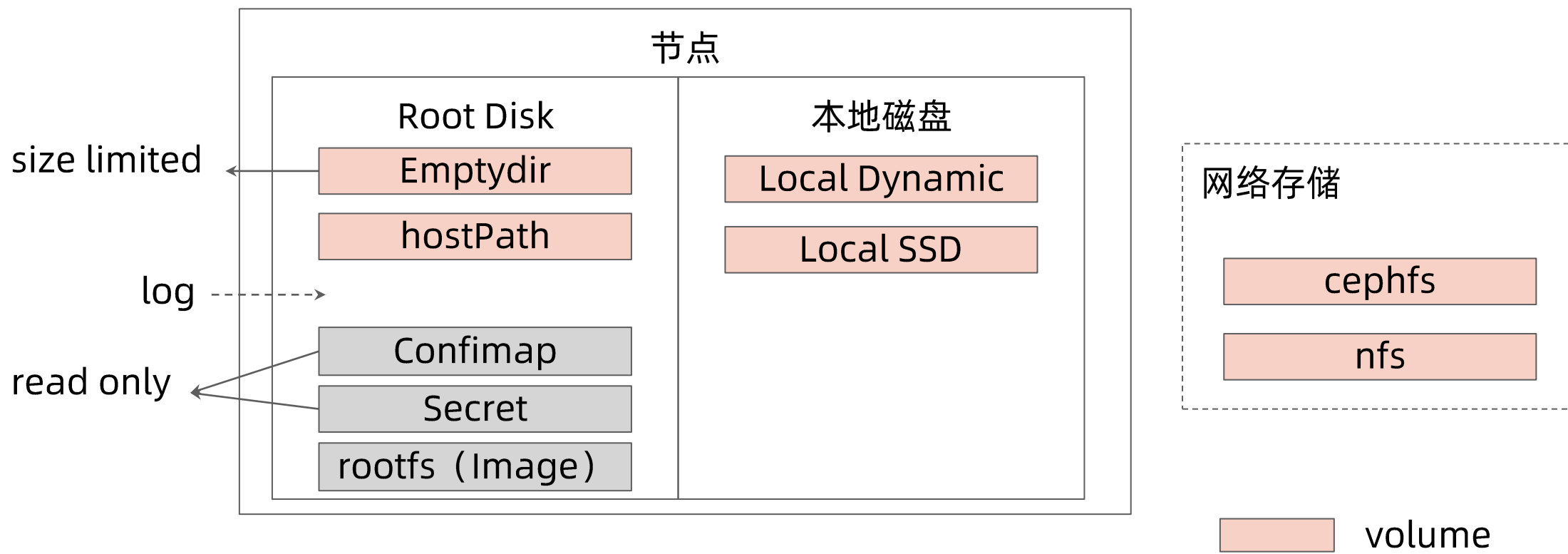
在 Kubernetes 上部署应用的挑战

资源规划

- 每个实例需要多少计算资源
 - CPU/GPU?
 - Memory
- 超售需求
- 每个实例需要多少存储资源
 - 大小
 - 本地还是网盘
 - 读写性能
 - Disk IO
- 网络需求
 - 整个应用总体 QPS 和带宽

Pod 的数据管理

- local-ssd: 独占的本地磁盘，独占 IO，固定大小，读写性能高。
- Local-dynamic: 基于 LVM，动态分配空间，效率低。



我的数据应该保存在哪里

存储卷类型	是应用进程重启后数据是否存在	Pod 重新调度后数据是否存在	容量限制	注意
emptydir	Yes	No	Yes	推荐，了解 emptydir 的 size limit 行为
hostPath	Yes	No	No	需要额外全权，不建议普通应用使用，多开放给节点管理员
local ssd/dynamic	Yes	No	Yes. Quota	无备份
网络存储	Yes	Yes	Yes. Quota	依赖与网络存储稳定性
容器 rootfs	No	No	No	不要写数据

不要短时间内打印过多日志以防在日志滚动之前节点磁盘就被耗光

应用配置

传入方式

- Environment Variables
- Volume Mount

数据来源

- Configmap
- Secret
- Downward API

高可用部署

需要多少实例？

如何控制失败域，部署在几个地区，AZ，集群？

如何进行精细的流量控制？

如何做按地域的顺序更新？

如何回滚？

如何应对基础架构的影响

系统管理员	应用	类型	影响	建议
自主中断	非自主中断	Kubelet 升级	大部分情况容器不会受影响	
			- 某些版本更新会重建 Container - Pods 会在秒级重建，业务恢复速度依赖应用启动速度	- 多实例部署 - 故障域控制，跨节点跨机架部署
		Kubelet 或者运行时无响应	- 如果是短时无响应，应用不应该受影响 - 如果是长时间无响应，需要驱逐 Pod	- 故障域控制，跨节点跨机架部署 - 合理的健康探针 - 设置合理的 toleration seconds
		节点替换	- 节点会被 drain 掉，节点会被删除 - Pod 会被驱逐	- 故障域控制，跨节点跨机架部署 - 使用 PDB 与基础架构约定驱逐策略 - 使用 preStop 备份关键数据
		节点重启	Pod 会失效数分钟	- 故障域控制，跨节点跨机架部署 - 设置合理的 toleration seconds
非自主中断		节点 Crash	- Pod 会在15分钟后被驱逐 - Pod 会失效15分钟以上	故障域控制，跨节点跨机架部署

PodDisruptionBudget

PDB 是为了自主中断时保障应用的高可用。

在使用 PDB 时，你需要弄清楚你的应用类型以及你想要的应对措施：

- 无状态应用：
 - 目标：至少有60%的副本 Available。
 - 方案：创建 PDB Object，指定 minAvailable 为60%，或者 maxUnavailable 为40%。
- 单实例的有状态应用：
 - 目标：终止这个实例之前必须提前通知客户并取得同意。
 - 方案：创建 PDB Object，并设置 maxUnavailable 为0。
- 多实例的有状态应用：
 - 目标最少可用的实例数不能少于某个数 N， 例如 etcd。
 - 方案：设置 maxUnavailable=1或者 minAvailable=N， 分别允许每次只删除一个实例和每次删除 $\text{expected_replicas} - \text{minAvailable}$ 个实例。

基础架构与应用团队的约束

基础架构团队在移除一个节点时，应遵循如下流程

- 将node置为不可调度

```
kubectl cordon <node name>
```

- 执行node drain排空节点，将其上运行的 Pod 平滑迁移至其他节点

```
kubectl drain <node name>
```

应用开发人员针对敏感应用，可定义 PDB 来确保应用不会被意外中断

```
apiVersion:
policy/v1beta1

kind:
PodDisruptionBudget

metadata:
  name: zk-pdb

spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: zookeeper
```

部署方式

多少实例

更新策略

- MaxSurge
- MaxUnavailable（需要考虑 ResourceQuota 的限制）

深入理解 PodTemplateHash 导致的应用的易变性

服务发布

需要把服务发布至集群内部或者外部，服务的不同类型：

- ClusterIP (Headless)
- NodePort
- LoadBalancer
- ExternalName

证书管理和七层负载均衡的需求

需要 gRPC 负载均衡如何做？

DNS 需求

与上下游服务的关系

服务发布的挑战

kube-dns

- DNS TTL 问题

Service

- ClusterIP 只能对内
- Kube-proxy 支持的 iptables/ipvs 规模有限
- IPVS 的性能和生产化问题
- kube-proxy 的 drift 问题
- 频繁的 Pod 变动 (spec change, failover, crashLoop) 导致 LB 频繁变更
- 对外发布的 Service 需要与企业 ELB 即成
- 不支持 gRPC
- 不支持自定义 DNS 和高级路由功能

Ingress

- Spec 要 deprecate

其他可选方案?

无状态应用管理

Replicaset 副本集

- 用什么 Pod 模版创建多少个实例。
- replicas: 2

Deployment

- 描述的是部署过程

- 版本管理

annotations:

deployment.kubernetes.io/revision: "1"

spec:

revisionHistoryLimit: 10

- 滚动升级策略

strategy:

rollingUpdate:

maxSurge: 25%

maxUnavailable: 1

type: RollingUpdate

有状态应用管理

Statefulset

与 deployment 相比, 多了

serviceName: nginx-ss

Volume claim template

volumeClaimTemplates:

- metadata:

- creationTimestamp: null

- name: www

- spec:

- accessModes:

- ReadWriteOnce

- resources:

- requests:

- storage: 1Gi

- volumeMode: Filesystem

有状态应用-Operator

创建 Operator 的关键是 CRD（自定义资源）的设计。

- Kubernetes 对象是可扩展的，扩展的方式有

- 基于原生对象

- 生成 types 对象，并通过 client-go 生成相应的 clientset, lister, informer。
- 实现对象的 registry backend，即定义对象任何存储进 etcd。
- 注册对象的 scheme 至 apiserver。
- 创建该对象的 apiservice 生命，注册该对象所对应的 api handler。

基于原生对象往往需要通过 aggregation apiserver 把不同对象组合起来。

- 基于 CRD

- 在不同应用业务环境下，对于平台可能有一些特殊的需求，这些需求可以抽象为 Kubernetes 的扩展资源，而 Kubernetes 的 CRD（CustomResourceDefinition）为这样的需求提供了轻量级的机制，保证新的资源的快速注册和使用。
- 在更老的版本中，TPR（ThirdPartyResource）是与 CRD 类似的概念，但是在1.9以上的版本中被弃用，而 CRD 则进入的 beta 状态。

如何使用 CRD

用户向 Kubernetes API 服务注册一个带特定 schema 的资源，并定义相关 API

- 注册一系列该资源的实例
- 在 Kubernetes 的其它资源对象中引用这个新注册资源的对象实例
- 用户自定义的 controller 例程需要对这个引用进行释义和实施，让新的资源对象达到预期的状态

基于 CRD 的开发过程

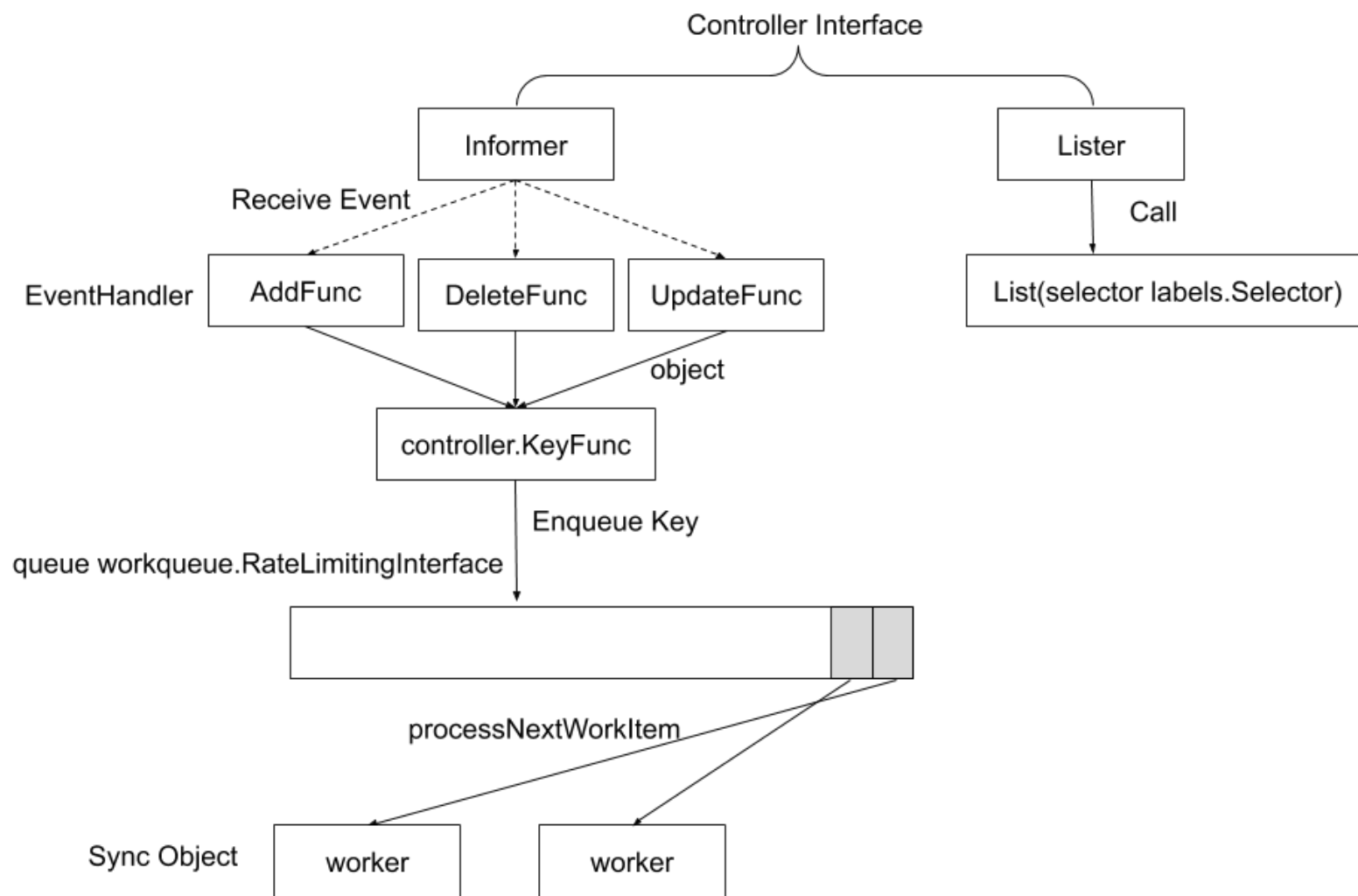
借助 Kubernetes RBAC 和 authentication 机制来保证该扩展资源的 security、access control、authentication 和 multitenancy。

将扩展资源的数据存储到 Kubernetes 的 etcd 集群。

借助 Kubernetes 提供的 controller 模式开发框架，实现新的 controller，并借助 APIServer 监听 etcd 集群关于该资源的状态并定义状态变化的处理逻辑。

该功能可以让开发人员扩展添加新功能，更新现有的功能，并且可以自动执行一些管理任务，这些自定义的控制器就像 Kubernetes 原生的组件一样，Operator 直接使用 Kubernetes API 进行开发，也就是说他们可以根据这些控制器内部编写的自定义规则来监控集群、更改 Pods/Services、对正在运行的应用进行扩缩容。

控制器模式



控制器代码示例

```
var shareFactory informers.SharedInformerFactory
if targetNs == "" {
    shareFactory = informers.NewSharedInformerFactory(k8sclient, DefaultResync)
} else {
    shareFactory = informers.NewFilteredSharedInformerFactory(k8sclient, DefaultResync, targetNs, nil)
}
```

```
podInformer := shareFactory.Core().V1().Pods()
podInformer.Informer().AddEventHandler(cache.ResourceEventHandlerFuncs{
    AddFunc:    addPod,
    UpdateFunc: updatePod,
    DeleteFunc: deletePod,
})
```

```
func addPod(obj interface{}) {
    pod, _ := obj.(*apiv1.Pod)
    enqueue(pod)
}
```

```
func updatePod(old, cur interface{}) {
    pod, _ := cur.(*apiv1.Pod)
    enqueue(pod)
}
```

```
func deletePod(obj interface{}) {
    pod, _ := obj.(*apiv1.Pod)
    enqueue(pod)
}

func processNextWorkItem() bool {
    key, quit := ac.queue.Get()
    if quit {
        return false
    }
    defer ac.queue.Done(key)

    err := syncPod(key.(string))
    ac.handleErr(err, key)
    return true
}

func syncPod(key string) error {
    // process logic
}
```


安装 kubebuilder

下载至本地

参考链接: <https://github.com/kubernetes-sigs/kubebuilder/releases>

```
mv <your_download_binary> /usr/local/bin/kubebuilder
```

Kubebuilder

辅助生成 CRD

- `kubebuilder init --domain example.com`
- `kubebuilder create api --group infra --version v1 --kind WebService`

修改 Spec

Make install 更新 CRD 定义

创建项目

运行下面的命令创建项目

```
kubebuilder init --domain myapps.cncamp.io
```

该命令生成如下文件

- `go.mod`: 依赖管理
- `Makefile`: 编译文件
- `PROJECT`: Kubebuilder 为生成新组建的元数据配置

为项目添加新 API

```
kubebuilder create api --group myapp.cncamp.io --version v1 --kind HTTPClient
```

修改API代码: 101/appsonk8s/kubebuilder/api/v1/groupversion_info.go

```
type HTTPClientSpec struct {
```

```
// INSERT ADDITIONAL SPEC FIELDS - desired state of cluster
```

```
// Important: Run "make" to regenerate code after modifying this file
```

```
// Foo is an example field of HTTPClient. Edit httpclient_types.go to remove/update
```

```
ServerAddress string `json:"serverAddress,omitempty"`
```

```
count int `json:"count,omitempty"`
```

```
}
```

```
// HTTPClientStatus defines the observed state of HTTPClient
```

```
type HTTPClientStatus struct {
```

```
Succeeded int `json:"succeeded,omitempty"`
```

```
Failed int `json:"failed,omitempty"`
```

```
// INSERT ADDITIONAL STATUS FIELD - define observed state of cluster
```

```
// Important: Run "make" to regenerate code after modifying this file
```

```
}
```

更新 CRD

```
make install
```

```
/Users/fmeng/go/src/github.com/cncamp/101/appsonk8s/kubebuilder/bin/controller-gen "crd:trivialVersions=true,preserveUnknownFields=false" rbac:roleName=manager-role webhook paths="./..." output:crd:artifacts:config=config/crd/bases
```

```
go: creating new go.mod: module tmp
```

修改控制器代码并运行

参考链接：

<https://github.com/cncamp/httpclient>

make run

创建 httpclient 对象

有状态应用的复杂性讨论

有状态应用部署示例-mysql

1. 高可用部署

- 构建 Galera cluster, 提供多活高可用 mysql 集群
- 多实例跨集群/跨机架/跨主机

2. 持久化存储

- 需要为每个 Pod 创建 PVC 并 mount
- 读写性能保证
- local dynamic 作为数据盘, cephfs 作为备份盘

3. 有状态应用的复杂配置

- 与无状态应用不一样, mysql 需要复杂配置以完成 galera 集群的构建
/etc/mysql/conf.d/galera.cnf

配置细节

```
# Configuration the First Node(Primary Component)
[mysqld]
binlog_format=ROW
default-storage-engine=innodb
innodb_autoinc_lock_mode=2
bind-address=0.0.0.0
# Galera Provider Configuration
wsrep_on=ON
wsrep_provider=/usr/lib/galera/libgalera_smm.so
# Galera Cluster Configuration
wsrep_cluster_name="test_cluster"
wsrep_cluster_address="gcomm://First_Node_IP,Second_Node_IP,Third_Node_IP"
# Galera Synchronization Configuration
wsrep_sst_method=rsync
# Configuration on all Galera Nodes
wsrep_node_address="This_Node_IP"
wsrep_node_name="This_Node_Name"
```

启动顺序

在 Primary Component 节点上运行

- mysqld_bootstrap

在其他节点上运行

- systemctl start mysql

发生了什么？

- 当节点第一次启动时，会自动生成 UUID 以代表当前节点身份
- 启动后，garala 会在数据目录生成 gvwstate.dat 文件，该文件内容记录 Primary Component 的 UUID 以及连接到当前 Primary Component 的节点的 UUID
- 如果 Primary Component 出现故障，则剩余节点会重新选择新的 Primary Component
- 若该文件已经存在，则无需额外执行 bootstrap 命令启动 Primary Compont，可依此规则编写 Operator 在多个节点构建此文件

```
my_uuid: d3124bc8-1605-11e4-aa3d-ab44303c044a
#vwbeg
view_id: 3 0dae1307-1606-11e4-aa94-5255b1455aa0 12
bootstrap: 0
member: 0dae1307-1606-11e4-aa94-5255b1455aa0 1
member: d3124bc8-1605-11e4-aa3d-ab44303c044a 1
#vwend
```

健康检查

mysql 提供健康检查 API

- 检查集群成员是否能接受查询请求
 - `SHOW GLOBAL STATUS LIKE 'wsrep_ready';`
- 检查节点是否与其他节点网络互通
 - `SHOW GLOBAL STATUS LIKE 'wsrep_connected';`
- 检查节点自上次查询结束后接收到的查询请求数量，如果结果为非0，意味着写请求不能立即处理
 - `SHOW STATUS LIKE 'wsrep_local_recv_queue_avg';`

健康检查应该影响 Pod 的 readiness probe，在进行版本升级时，确保大多数集群节点状态一致。

数据备份

推荐为 mysql 创建不同类型的 volume

- Local volume 用来做数据盘
- Network volume 用来做数据备份

创建 cronjob, 每天将数据备份至 backup 目录

- 备份文件为

一键恢复能力

- 导入备份目录的 sql file

版本发布和故障转移

针对配置了 Local disk 的 Pod, 当发生因版本变更而引发的 Pod 重建时, 新 Pod 在进行调度时, 调度器会查询 Pod 挂载的 volume 所在节点, 并将新 Pod 优先调度至该节点, 此场景不涉及到数据恢复。

其开销与 mysql 进程重启相差不大。

版本发布和故障转移

如果节点出现故障，如硬件故障，Kubernetes 的 Evict Manager 会将该 Pod 从故障节点驱逐，Operator 应确保新 Pod 会被重新构建。而新 Pod 会被调度至新节点，此场景等价于替换 mysql 中的少数节点。

- galara 集群中的少数 mysql 节点替换，不涉及到数据迁移，galara 会确保新节点的数据同步

若整个 mysql 集群需要做数据恢复，则应该从 backup 目录对应的网络 volume 恢复数据。此时可选择：

- 只恢复单一节点数据：配置简单
- 恢复所有节点数据：恢复速度快

与基础架构的 Contract

- PodDisruptionBudget

业界高可用数据库方案分享

参考链接: <https://cncamp.notion.site/mysql-on-kubernetes-eb043ee841dc41618ce4798bca187bbf>

2. Spec 管理神器-Helm

什么是 Helm

Helm 特性

- Helm chart 是创建一个应用实例的必要的配置组，也就是一堆 Spec。
- 配置信息被归类为模版（Template）和值（Value），这些信息经过渲染生成最终的对象。
- 所有配置可以被打包进一个可以发布的对象中。
- 一个 release 就是一个有特定配置的 chart 的实例。

Helm 的组件

Helm client

- 本地 chart 开发
- 管理 repository
- 管理 release
- 与 helm library 交互
 - 发送需要安装的 chart
 - 请求升级或着卸载存在的 release

Helm library

- 负责与 APIServer 交互，并提供以下功能
 - 基于 chart 和 configuration 创建一个 release
 - 把 chart 安装进 kubernetes，并提供相应的 release 对象
 - 升级和卸载
 - Helm 采用 Kubernetes 存储所有配置信息，无需自己的数据库

Kubernetes Helm 架构

Helm 的目标

从头创建 chart

把 chart 打包成压缩文件 (tgz)

与 chart 的存储仓库交互 (chart repository)

Kubernetes 集群中的 chart 安装与卸载

管理用 Helm 安装的 release 的生命周期

Helm 的安装

下载

参考链接: <https://github.com/helm/helm/releases>

安装

- 解压
- `mv <your_downloaded_binary> /usr/local/bin/helm`

Helm chart 的基本使用

创建一个 chart

```
helm create myapp
```

myapp/

Chart.yaml # 包含了 chart 信息的 YAML 文件

values.yaml # chart 默认的配置值

charts/ # 包含 chart 依赖的其他 chart

templates/ # 模板目录，当和 values 结合时，可生成有效的 Kubernetes manifest 文件

复用已存在的成熟 Helm release

- 针对 Helm release repo 的操作

```
helm repo add grafana https://grafana.github.io/helm-charts
```

```
helm repo update
```

```
helm repo list
```

```
helm search repo grafana
```

- 从 remote repo 安装 Helm chart

```
helm upgrade --install loki grafana/loki-stack
```

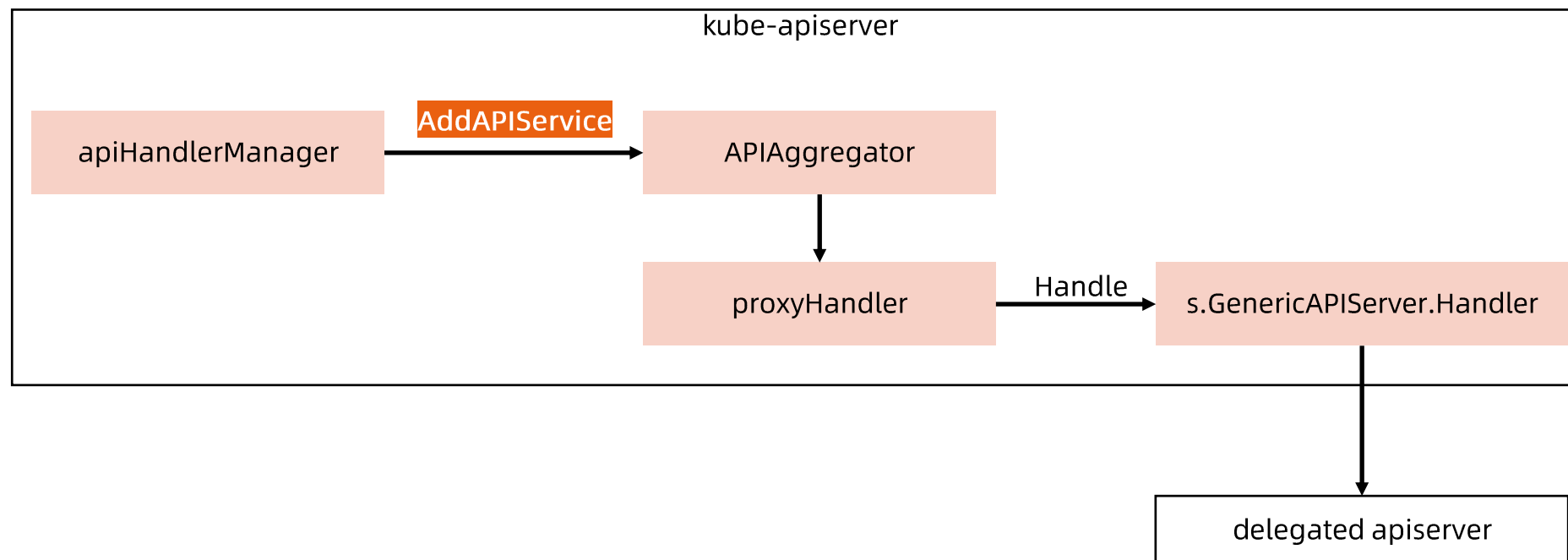
下载并从本地安装 helm chart

- `helm pull grafana/loki-stack`

```
helm upgrade --install loki ./loki-stack
```

3. metrics-server

Aggregated APIServer



Metrics-Server

`metrics-server` 是 Kubernetes 监控体系中的核心组件之一，它负责从 kubelet 收集资源指标，然后对这些指标监控数据进行聚合（依赖 kube-aggregator），并在 Kubernetes Apiserver 中通过 Metrics API（`/apis/metrics.k8s.io/`）公开暴露它们，但是 `metrics-server` 只存储最新的指标数据（CPU/Memory）。

你的 kube-apiserver 要能访问到 metrics-server;

需要 kube-apiserver 启用聚合层;

组件要有认证配置并且绑定到 metrics-server;

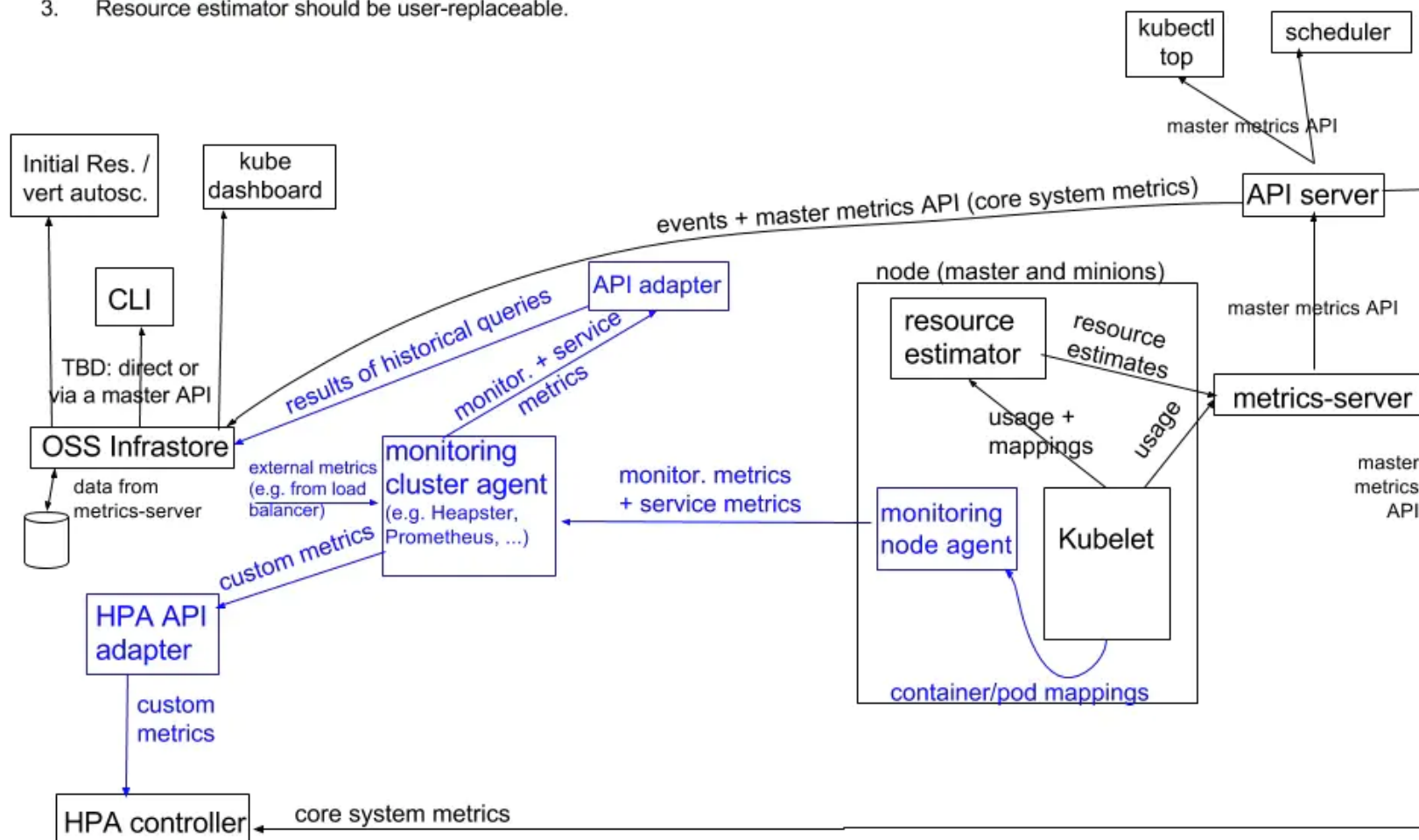
Pod / Node 指标需要由 Summary API 通过 kubelet 公开。

Monitoring architecture proposal: OSS

(arrows show direction of metrics flow)

Notes

1. Arrows show direction of metrics flow.
2. **Monitoring pipeline is in blue.** It is user-supplied and optional.
3. Resource estimator should be user-replaceable.



metrics-server 的本质

将指标数据转换成metrics.k8s.io的api调用返回值

```
k get --raw "/api/v1/nodes/node1/proxy/metrics/resource"
```

```
k top node
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
node1	294m	7%	3003Mi	25%

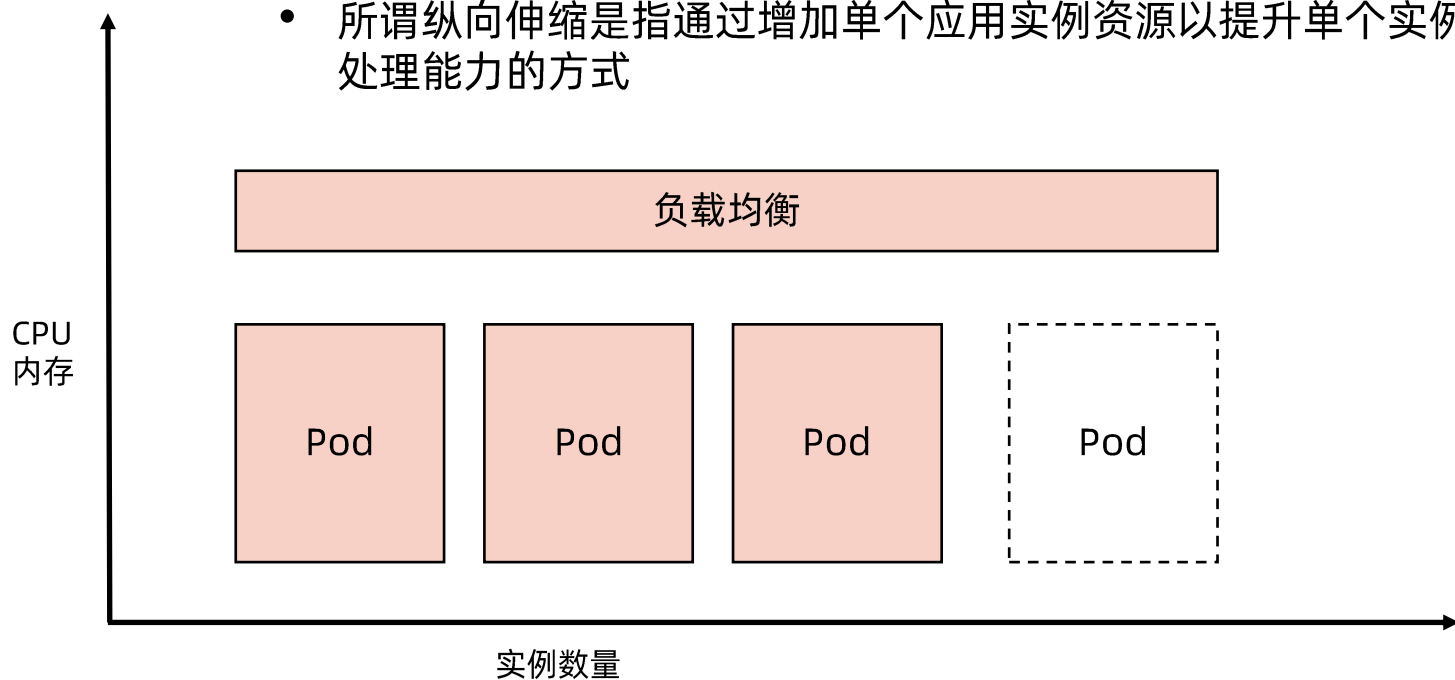
```
k top pod
```

NAME	CPU(cores)	MEMORY(bytes)
centos-5fdd4bb694-bl62g	3m	3Mi
httpserver-deployments-7df744646-jclhz	1m	5Mi
nginx-deployment-6799fc88d8-6pmrt	0m	5Mi
nginx-deployment-6799fc88d8-qv5f4	0m	5Mi
nginx-deployment-6799fc88d8-zkf9d	0m	5Mi

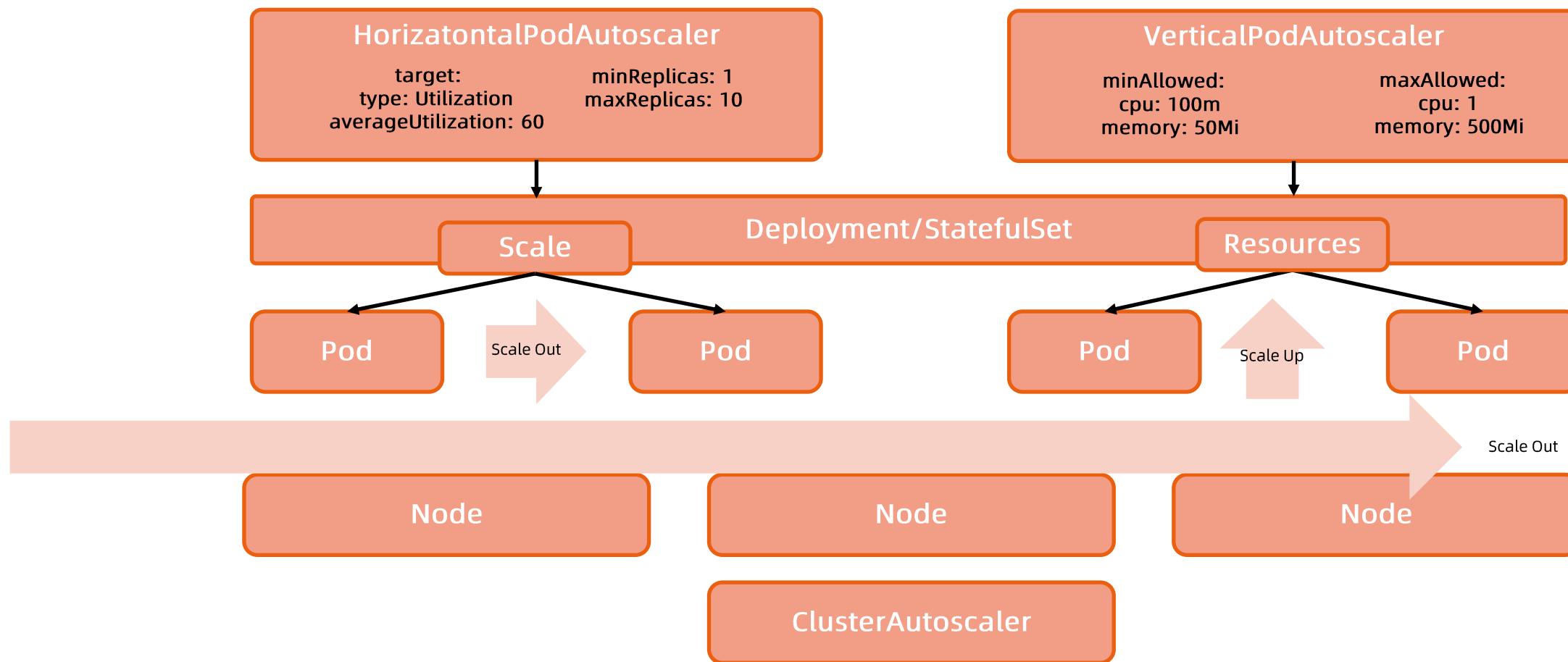
4. 自动扩容缩容-HPA

横向伸缩和纵向伸缩

- 应用扩容是指在应用接收到的并发请求已经处于其处理请求极限边界的情形下，扩展处理能力而确保应用高可用的技术手段
- Horizontal Scaling
 - 所谓横向伸缩是指通过增加应用实例数量分担负载的方式来提升应用整体处理能力的方式
- Vertical Scaling
 - 所谓纵向伸缩是指通过增加单个应用实例资源以提升单个实例处理能力，进而提升应用整体处理能力的方式



理解云原生的弹性能力



HPA

- HPA (Horizontal Pod Autoscaler) 是 Kubernetes 的一种资源对象，能够根据某些指标对在 statefulSet、replicaSet、deployment 等集合中的 Pod 数量进行横向动态伸缩，使运行在上面的服务对指标的变化有一定的自适应能力。
- 因节点计算资源固定，当 Pod 调度完成并运行以后，动态调整计算资源变得较为困难，因为横向扩展具有更大优势，HPA 是扩展应用能力的第一选择。
- 多个冲突的 HPA 同时创建到同一个应用的时候会有无法预期的行为，因此需要小心维护 HPA 规则。
- HPA 依赖于 Metrics-Server。

HPA Spec

```
apiVersion: autoscaling/v2beta2
```

```
kind: HorizontalPodAutoscaler
```

```
metadata:
```

```
  name: mynginx
```

```
spec:
```

```
  # HPA 的伸缩对象描述，HPA会动态修改该对象的 Pod 数量
```

```
  scaleTargetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: mynginx
```

```
  # HPA 的最小 Pod 数量和最大 Pod 数量
```

```
  minReplicas: 1
```

```
  maxReplicas: 3
```

```
  # 监控的指标数组，支持多种类型的指标共存
```

```
  metrics:
```


HPA 支持的指标类型

对于按 Pod 统计的资源指标（如 CPU），控制器从资源指标 API 中获取每一个 **HorizontalPodAutoscaler 指定的 Pod 的度量值**，如果设置了目标使用率，控制器获取每个 Pod 中的容器资源使用情况，并计算资源使用率。如果设置了 target 值，将直接使用原始数据（不再计算百分比）。

如果 Pod 使用自定义指示，控制器机制与资源指标类似，区别在于自定义指标只使用原始值，而不是使用率。

如果 Pod 使用对象指标和外部指标（每个指标描述一个对象信息）。这个指标将直接根据目标设定值相比较，并生成一个上面提到的扩缩比例。在 **autoscaling/v2beta2 版本 API 中**，这个指标也可以根据 Pod 数量平分后再计算。

HPA 指标

Resource 类型的指标

- type: Resource

resource:

name: cpu

Utilization 类型的目标值, Resource 类型的指标只支持 Utilization 和 AverageValue 类型的目标值

target:

type: Utilization

averageUtilization: 50

Pods 类型的指标

- type: Pods

Pods:

metric:

name: packets-per-second

AverageValue 类型的目标值, Pods 指标类型下只支持 AverageValue 类型的目标值

target:

type: AverageValue

averageValue: 1k

算法细节

HPA 算法非常简单

期望副本数 = $\text{ceil}[\text{当前副本数} * (\text{当前指标} / \text{期望指标})]$

当前度量值为 200m，目标设定值为 100m，那么由于 $200.0/100.0 == 2.0$ ，副本数量将会翻倍。
如果当前指标为 50m，副本数量将会减半，因为 $50.0/100.0 == 0.5$ 。如果计算出的扩缩比例接近 1.0（根据 `--horizontal-pod-autoscaler-tolerance` 参数全局配置的容忍值，默认为 0.1），将会放弃本次扩缩。

滚动升级时扩缩

当你为一个 Deployment 配置自动扩缩时，你要为每个 Deployment 绑定一个 HorizontalPodAutoscaler。

HorizontalPodAutoscaler 管理 Deployment 的 replicas 字段。

Deployment Controller 负责设置下层 ReplicaSet 的 replicas 字段，以便确保在上线及后续过程副本个数合适。

想一想：为什么 deploymentSpec 中的 replicas 字段的类型为 *int，而不是 int？

冷却/延迟支持

当使用 Horizontal Pod Autoscaler 管理一组副本扩缩时，有可能因为指标动态的变化造成副本数量频繁的变化，有时这被称为 **抖动 (Thrashing)**。

--horizontal-pod-autoscaler-downscale-stabilization: 设置缩容冷却时间窗口长度。

水平 Pod 扩缩器能够记住过去建议的负载规模，并仅对此时间窗口内的最大规模执行操作。

默认值是 5 分钟 (5m0s)。

扩缩策略

在 Spec 字段的 behavior 部分可以指定一个或多个扩缩策略。当指定多个策略时，默认选择允许更改最多的策略。下面的例子展示了缩容时的行为：

```
behavior:
  scaleDown:
    policies:
      - type: Pods
        value: 4
        periodSeconds: 60
      - type: Percent
        value: 10
        periodSeconds: 60
```

HPA 练习

安装 metrics-server (本质上是一个 aggregated Server)

```
cd 101/appsonk8s/hpa/metrics-server
```

```
kubectl apply -f components.yaml
```

```
kubectl top node
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
k8snode	325m	5%	2148Mi	18%

启动应用

- 创建 php server

```
cd 101/appsonk8s/hpa
```

```
kubectl apply -f php-apache.yaml
```

- 设置 HPA 规则

```
kubectl autoscale deployment php-apache --cpu-percent=50 --min=1 --max=3
```

- 查看 HPA Spec

```
spec:
```

```
  maxReplicas: 3
```

```
  minReplicas: 1
```

```
  scaleTargetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: php-apache
```

```
  targetCPUUtilizationPercentage: 50
```


测试 HPA

- 为服务器加压

```
kubectl run -i --tty load-generator --rm --image=busybox --restart=Never -- /bin/sh -c "while sleep 0.01; do wget -q -O- http://php-apache; done"
```

- 观测 top pod，可以发现当 pod cpu 利用率大于 500m 的时候，会创建更多 pod 出来分担压力
- 停止 load-generator 以后，等待一段时间，pod 数量会降为一个

```
watch kubectl top pod
```

k8snode: Sat Sep 11 09:52:05 2021

NAME	CPU(cores)	MEMORY(bytes)	
load-generator	12m	1Mi	
php-apache-6cd4b65f7b-4rwsq		240m	12Mi
php-apache-6cd4b65f7b-p878h		488m	12Mi

HPA 存在的问题

基于指标的弹性有滞后效应，因为弹性控制器操作的链路过长。

从应用负载超出阈值到 HPA 完成扩容之间的时间差包括：

- 应用指标数据已经超出阈值；
- HPA 定期执行指标收集滞后效应；
- HPA 控制 Deployment 进行扩容的时间；
- Pod 调度，运行时启动挂载存储和网络的时间；
- 应用启动到服务就绪的时间。

很可能在突发流量出现时，还没完成弹性扩容，既有的服务实例已经被流量击垮。

5. 自动扩容缩容-VPA

VPA

VPA 全称 Vertical Pod Autoscaler，即垂直 Pod 自动扩缩容，它根据容器资源使用率自动设置 CPU 和 内存的 requests，从而允许在节点上进行适当的调度，以便为每个 Pod 提供适当的资源。它既可以缩小过度请求资源的容器，也可以根据其使用情况随时提升资源不足的容量。

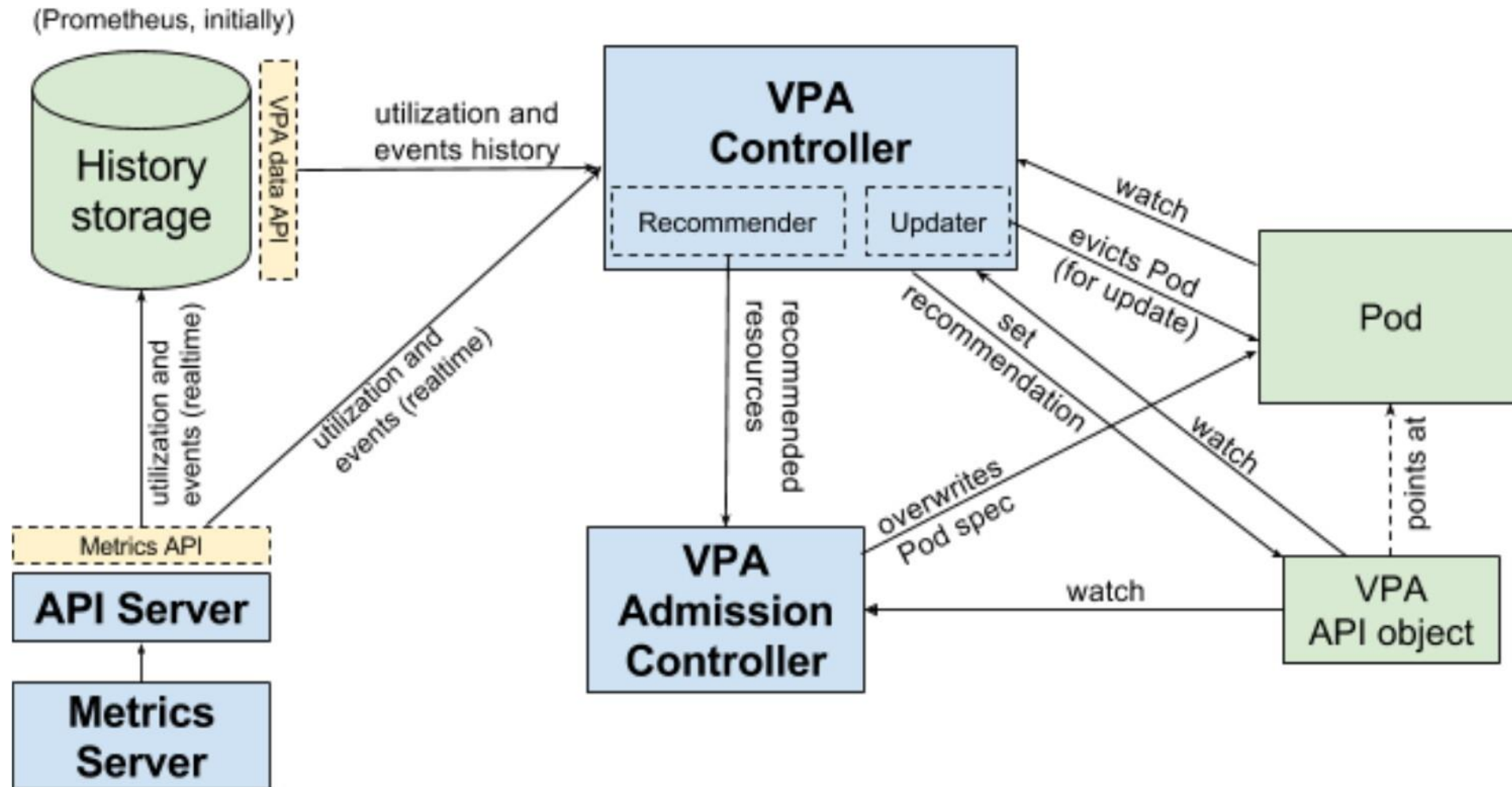
使用 VPA 的意义：

- Pod 资源用其所需，提升集群节点使用效率；
- 不必运行基准测试任务来确定 CPU 和内存请求的合适值；
- VPA 可以随时调整 CPU 和内存请求，无需人为操作，因此可以减少维护时间。

注意：

VPA 目前还没有生产就绪，在使用之前需要了解资源调节对应用的影响。

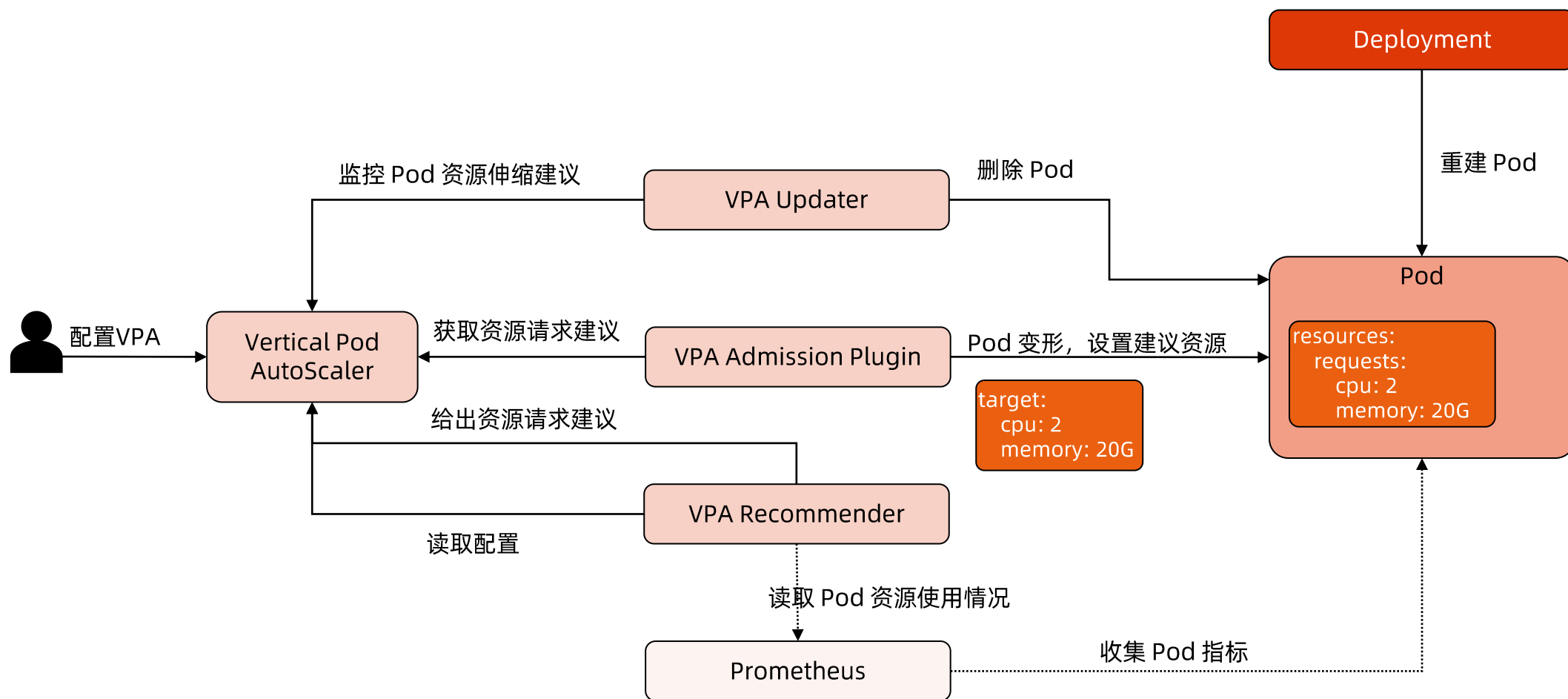
VPA 架构图



VPA 组件

- VPA 引入了一种新型的 API 资源：VerticalPodAutoscaler。
- VPA Recommender 监视所有 Pod，不断为它们计算新的推荐资源，并将推荐值存储在 VPA 对象中。它使用来自 Metrics-Server 的集群中所有 Pod 的利用率和 OOM 事件。
- 所有 Pod 创建请求都通过 VPA Admission Controller。
- VPA Updater 是负责 Pod 实时更新的组件。如果 Pod 在 "Auto" 模式下使用 VPA，则 Updater 可以决定使用推荐器资源对其进行更新。
- History Storage 是一个存储组件（如 Prometheus），它使用来自 API Server 的利用率信息和 OOM（与推荐器相同的数据）并将其持久存储。
- VPA 更新模式：
 - Off
 - Auto

VPA 工作原理

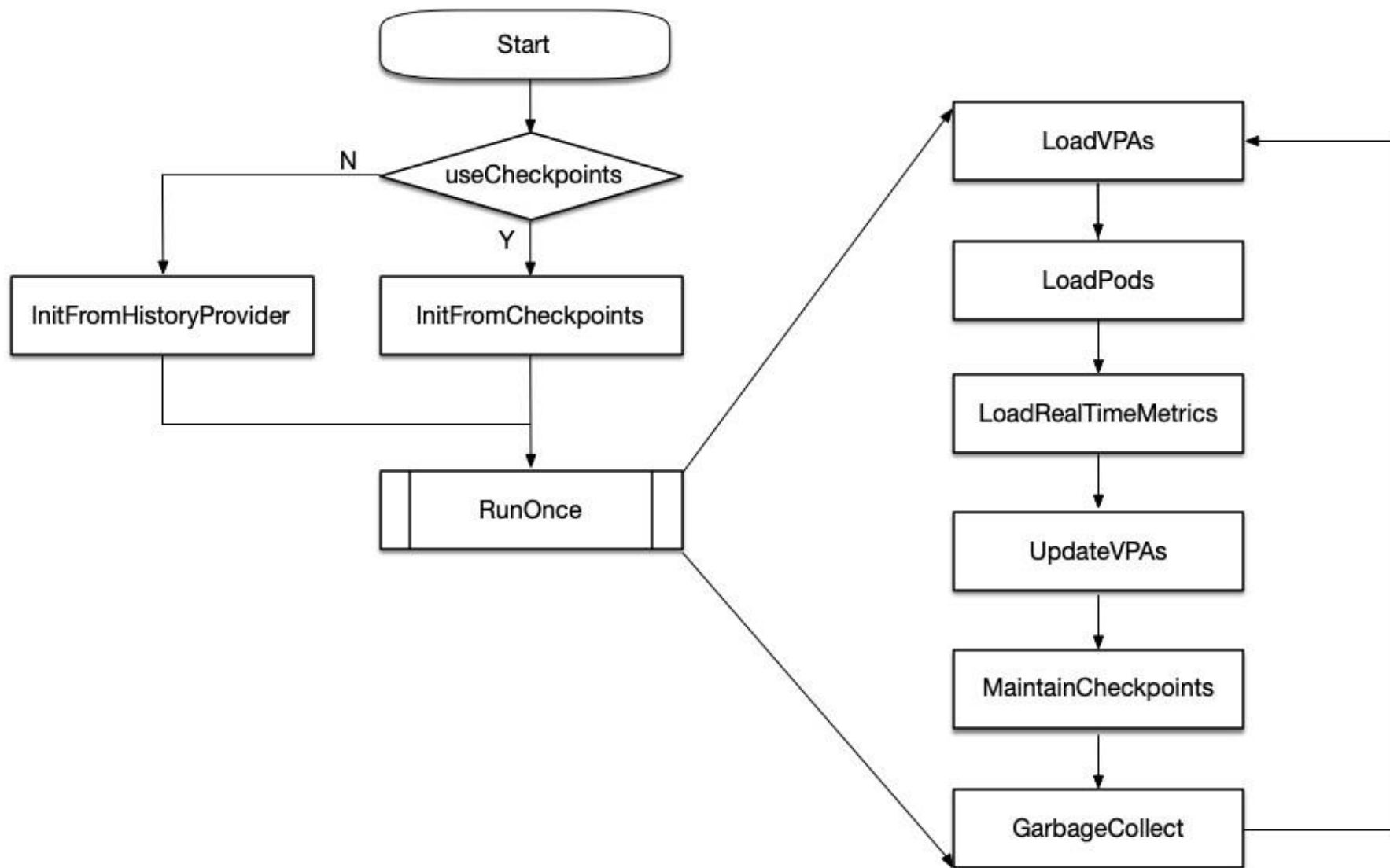


Recommender 设计理念

推荐模型（MVP）假设内存和 CPU 利用率是独立的随机变量，其分布等于过去 N 天观察到的变量（推荐值为 N=8 以捕获每周峰值）。

- 对于 CPU，目标是将容器使用率超过请求的高百分比（例如 95%）时的时间部分保持在某个阈值（例如 1% 的时间）以下。在此模型中，“CPU 使用率”被定义为在短时间间隔内测量的平均使用率。测量间隔越短，针对尖峰、延迟敏感的工作负载的建议质量就越高。最小合理分辨率为 1/min，推荐为 1/sec。
- 对于内存，目标是将特定时间窗口内容器使用率超过请求的概率保持在某个阈值以下（例如，24 小时内低于 1%）。窗口必须很长（ ≥ 24 小时）以确保由 OOM 引起的驱逐不会明显影响（a）服务应用程序的可用性（b）批处理计算的进度（更高级的模型可以允许用户指定 SLO 来控制它）。

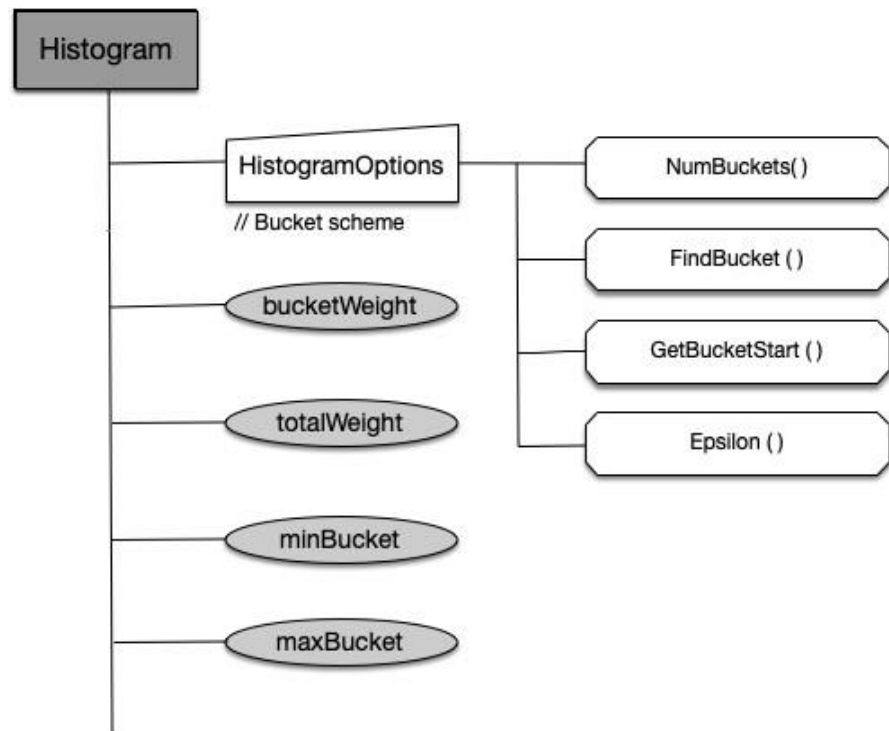
主要流程



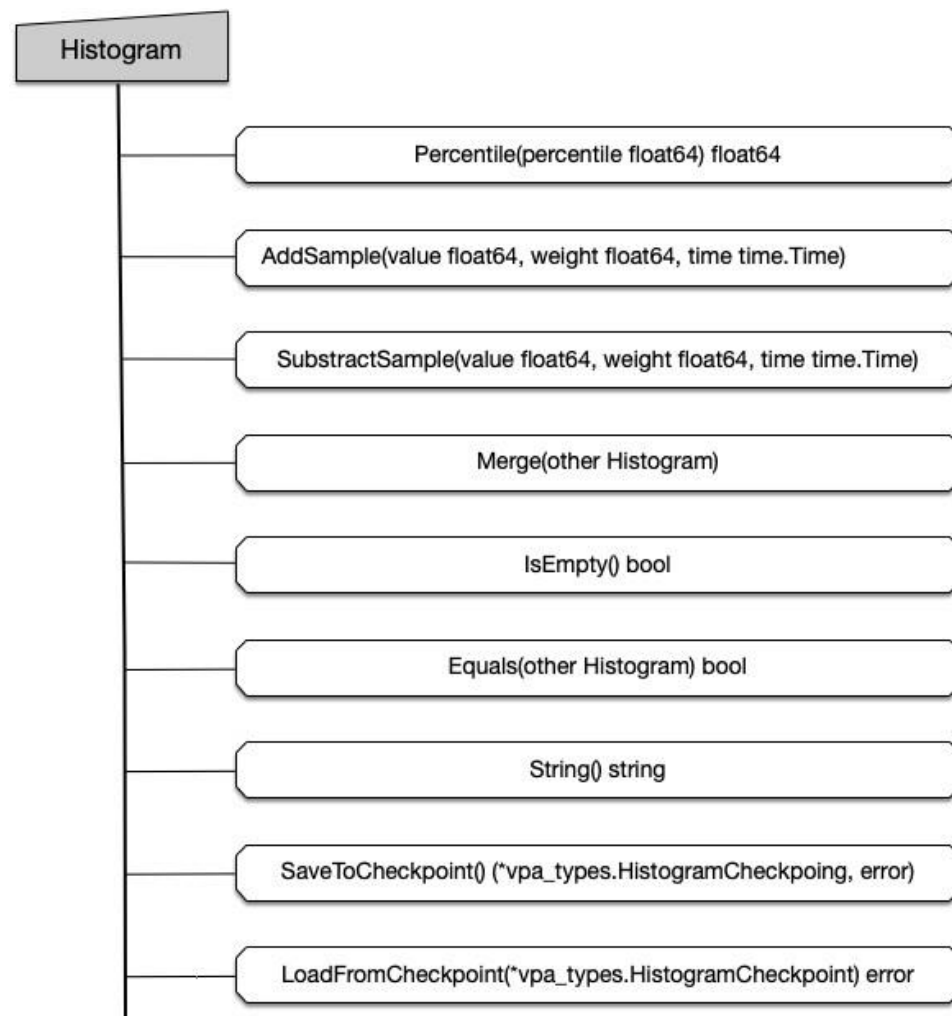
滑动窗口与半衰指数直方图

Recommender 的资源推荐算法主要受 Google AutoPilot moving window 推荐器的启发，假设 CPU 和 Memory 消耗是独立的随机变量，其分布等于过去 N 天观察到的变量分布（推荐值为 N=8 以捕获每周业务容器峰值）。

Recommender 组件获取资源消耗实时数据，存到相应资源对象 CheckPoint 中。CheckPoint CRD 资源本质上是一个直方图。



直方图统一对外提供的接口



直方图数据范围

Resource	minBucket	maxBucket	rate
cpu	0.01cores	1000 cores	5%
memory	10MB	1TB	5%

半衰期和权重系数

- 为每个样本数据权重乘上指数 $2^{((sampleTime - referenceTimestamp) / halfLife)}$ ，以保证较新的样本被赋予更高的权重，而较老的样本随时间推移权重逐步衰减。
- 默认情况下，每 24h 为一个半衰期，即每经过 24h，直方图中所有样本的权重（重要性）衰减为原来的一半。
- 当指数过大时，referenceTimestamp 就需要向前调整，以避免浮点乘法计算时向上溢出。
 - CPU 使用量样本对应的权重是基于容器 CPU request 值确定的。当 CPU request 增加时，对应的权重也随之增加。
 - 而 Memory 使用量样本对应的权重固定为 1.0。

VPA

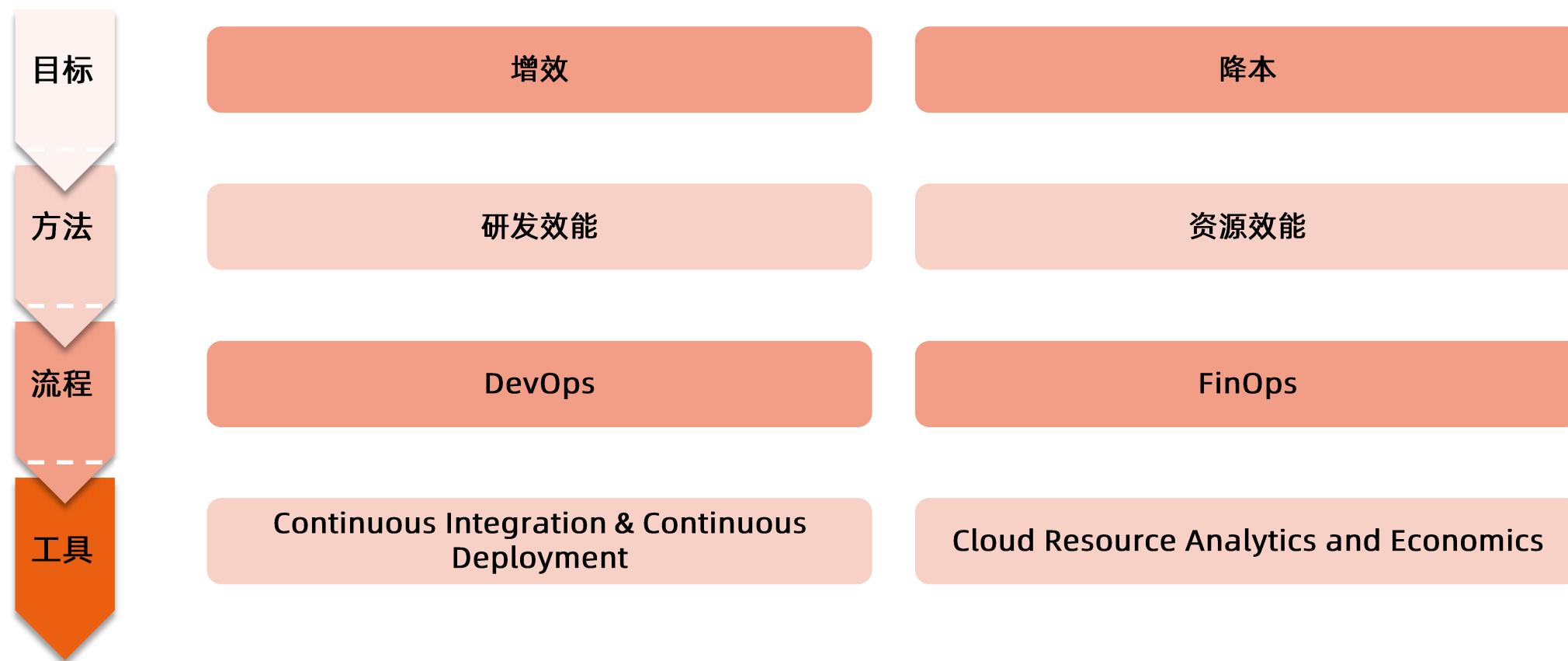
- VPA 的成熟度还不足
 - 更新正在运行的 Pod 资源配置是 VPA 的一项试验性功能，会导致 Pod 的重建和重启，而且有可能被调度到其他的节点上。
 - VPA 不会驱逐没有在副本控制器管理下的 Pod。目前对于这类 Pod，Auto 模式等同于 Initial 模式。
 - 目前 VPA 不能和监控 CPU 和内存度量的 Horizontal Pod Autoscaler (HPA) 同时运行，除非 HPA 只监控其他定制化的或者外部的资源度量。

VPA

- VPA 的成熟度还不足
 - VPA 使用 admission webhook 作为其准入控制器。如果集群中有其他的 admission webhook，需要确保它们不会与 VPA 发生冲突。准入控制器的执行顺序定义在 API Server 的配置参数中。
 - VPA 会处理出现的绝大多数 OOM (Out Of Memory) 的事件，但不保证所有的场景下都有效。
 - VPA 的性能还没有在大型集群中测试过。
 - VPA 对 Pod 资源 requests 的修改值可能超过实际的资源上限，例如节点资源上限、空闲资源或资源配额，从而造成 Pod 处于 Pending 状态无法被调度。同时使用集群自动伸缩 (ClusterAutoscaler) 可以一定程度上解决这个问题。
 - 多个 VPA 同时匹配同一个 Pod 会造成未定义的行为。

6. 如何解决社区基础弹性能力不足的问题

弹性的意义



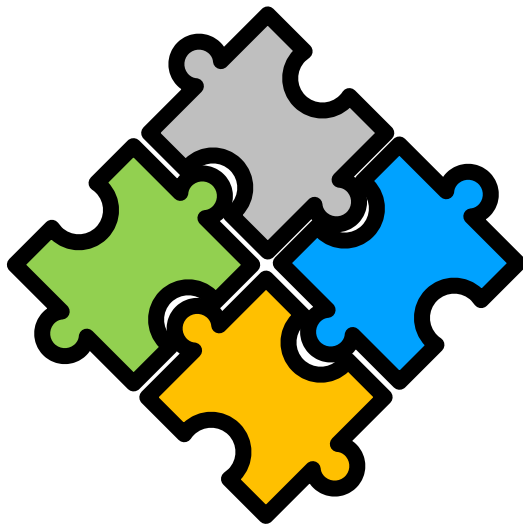
成本优化的关键路径

成本是业务稳定性跟资源利用率的矛盾

大多数稳定性都是由冗余换来的

成本是业务投入跟技术投入的矛盾

就一个人力，投入业务还是投入优化



理解成本

成本还是企业内不同组织/角色的矛盾

业务团队 vs 资源团队

云业务的去中心化部署和运维为成本管理带来挑战

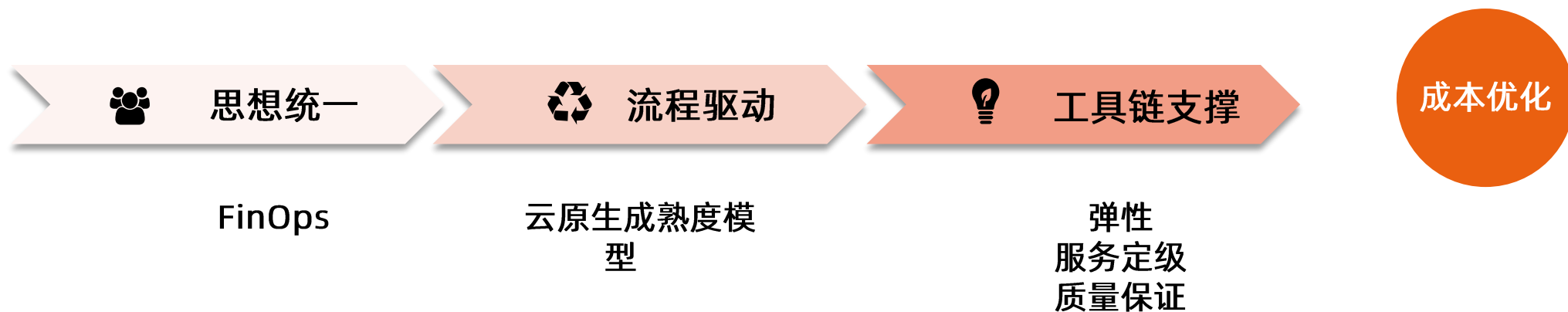
可以通过技术手段和组织手段减少矛盾

开启降本之路

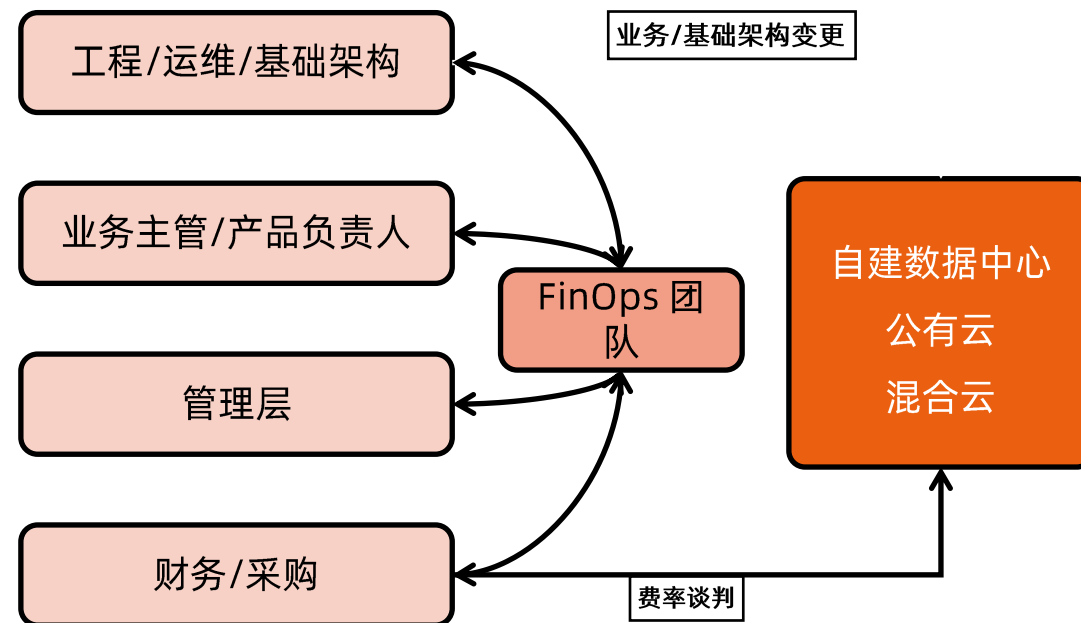
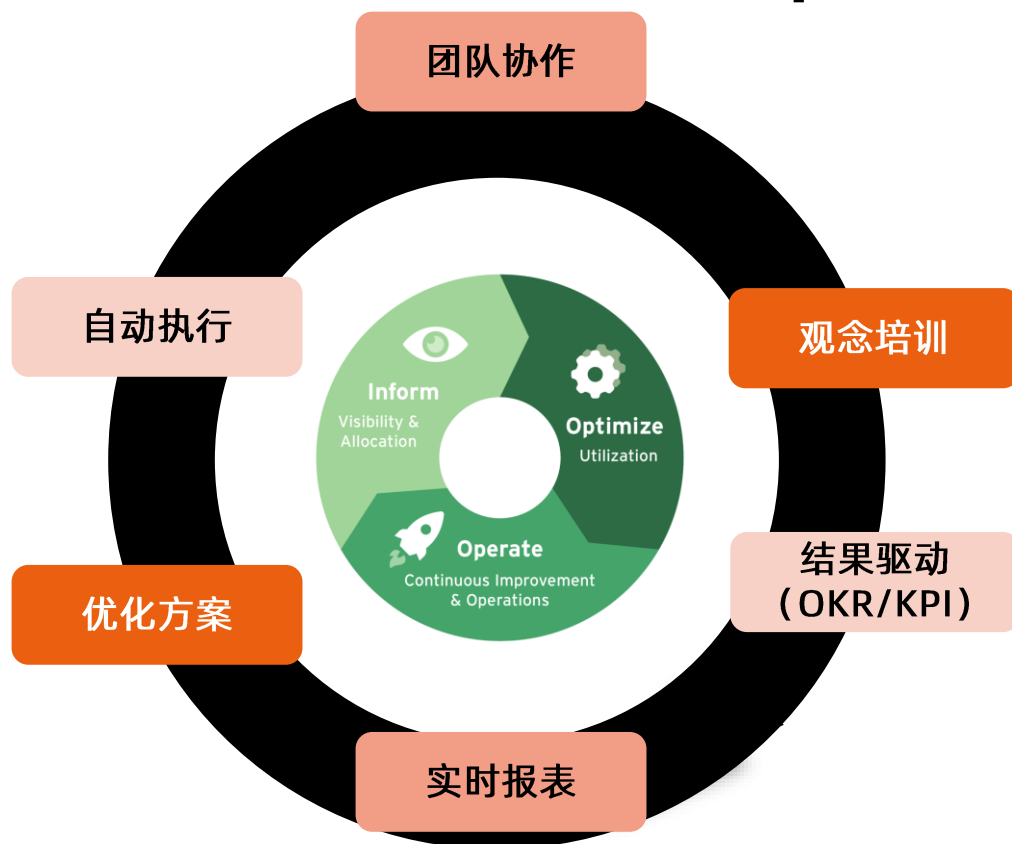
天时 - 云原生技术的日趋成熟

地利 - 统一上云

人和 - 思想统一



统一思想 - FinOps



- FinOps 基金会隶属于 Linux 基金会，致力推进企业的云资产管理
- FinOps 核心是确保企业在云中花费的每一分钱都获得最大价值

流程建设 - 云原生成熟度模型



业务侧 Workload 评分

指标	规则
Workload 利用率得分	以每天 Workload 的利用率 (used / request 总量) 峰值作为打分基础 1、利用率大于等于50%得100分 2、利用率小于50%: 分数 = 利用率 * 100 / 50 注: 峰值不去毛刺
Workload 弹性得分	• Workload 每天 Pod 数计算: 按10分钟聚合每个 Workload 的 Pod 个数 • Pod 信息按分钟上报, 每天1440个点, 取10分钟聚合数据作为Pod数据来规避偶尔的上报丢失情况。 • 如某天 Pod 数目发生变化, 且满足 $(\max - \min) > \max * 5\%$ 且 $(\max - \min) > 1$, 则本日弹性计数为1, 否则为0 1. 若30天内总计弹性计数 < 2, 则弹性得分记为0 2. 如果30天内 弹性计数=2, 则弹性得分记为60, 之后弹性计数每+1, 弹性得分+5, 满分100 3. 利用率得分90分以上, 弹性免考核, 弹性得分直接记为100 4. Pod 数小于3个的, 默认弹性置为60分
Workload 综合得分	单 Workload 综合得分 = (利用率得分 * 70% + HPA得分 * 30%) * 该 Workload 的 request / 该业务所有 Workload 总 request
业务综合得分	Sum (单 Workload 综合得分)

业务侧 Workload 评分 - 辅助指标

指标	说明	建议
Pod 伸缩比	该指标会计算各 Workload 的 limit 配置与 request 配置的比值。	建议业务根据实际使用情况，适当降低 request 配置、提升 limit 配置。从而提升 Pod 的伸缩比。
大核心占比	统计 Request 配置大于16核的 Pod 核心数占比	建议业务降低 Request 的配置，提升 Workload 的资源利用率及可调度性
容器平均核心	统计 Pod Request 配置平均核心数	建议业务降低 Request 的配置，提升 Workload 的资源利用率及可调度性
CPU: Memory	统计 CPU 与 Memory 的比值（Memory 单位为 GB）	建议业务降低 CPU 与 Memory 配比，提示资源利用效率
CPU: Memory 过高的占比	统计 CPU 与 Memory 的比值（Memory 单位为 GB）超过1:4的 Pod CPU占比	建议业务降低 CPU 与 Memory 配比，提示资源利用效率

平台集群评分

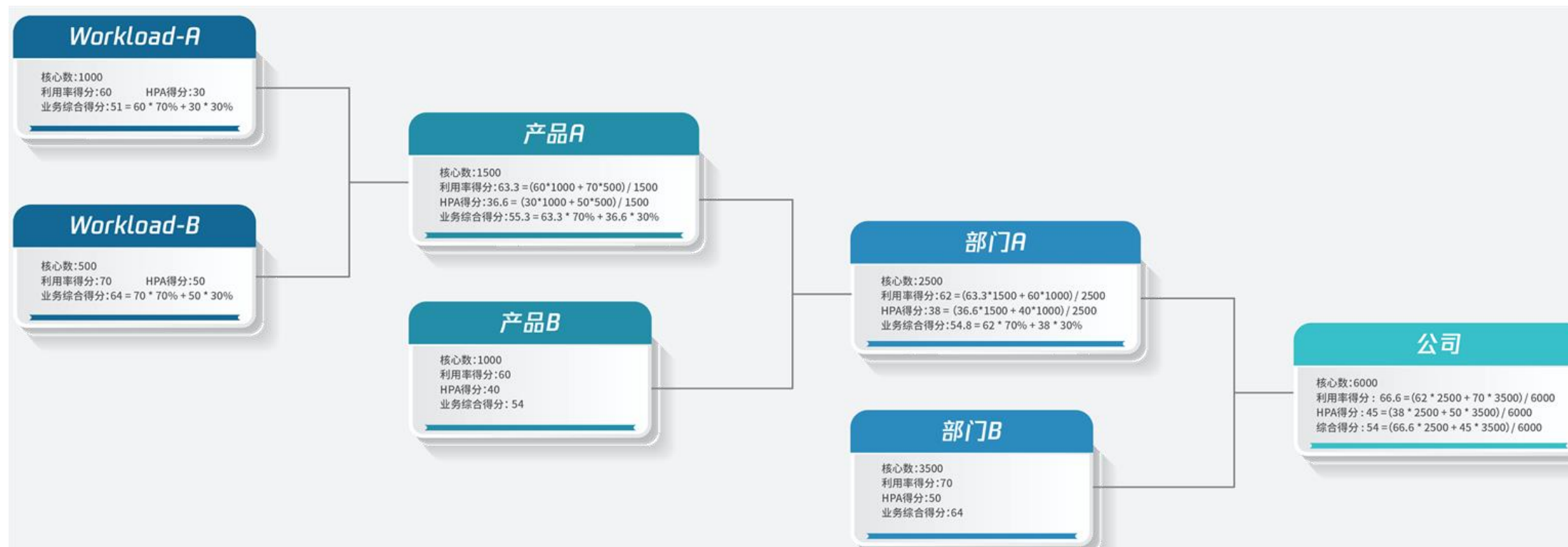
指标	规则
集群利用率得分	以每天集群的利用率峰值作为打分基础 1、利用率大于等于50%得100分； 2、利用率小于50%：分数 = 利用率 * 100 / 50 注：峰值不去毛刺
集群 CA 得分 CA (Cluster Autoscaler)	1、集群在云梯侧开启了 CA 功能可得60分。 2、1个月内集群节点数变化超过2天以上，每多变动1天加20分，满分100分。 3、每天集群最大节点数与最小节点数的差值大于等于3，记为一次CA。 上述得分规则未考虑集群的规模及实际利用率情况，在执行过程中发现了业务频繁刷分的现象。 补充规则，降低刷分行为对排行榜的影响。 $\text{当日CA得分} = 60 + \left(\frac{\text{昨日最小核心} - \text{当日最小核心}}{\text{昨日最小核心} * \frac{45 - \text{昨日周峰值利用率}}{45}} + \left(\frac{\text{当日周峰值利用率} - 36}{9} \right) \right) * 40$

辅助指标

指标	规则
超卖率	$\text{集群可分配核心数} / \text{集群机器总核心数}$
装箱率	$\text{集群所有 Workload Request CPU 总核心数} / \text{集群机器总核心数}$

成熟度指标计算公式

总成熟度得分 = 业务侧得分 * 50% + 平台侧得分 * 50%



驱动力

技术交流层面

业务上云经验研讨

外部云原生技术推广福利

团队及个人荣誉层面

资源优惠层面

奖项说明		半年度云原生标杆团队，将获得云原生特别激励奖		
卓越实践奖	最佳实践奖	优秀实践奖	年度自研上云奖	
3万	2万	1万	年底公布	

云成熟度模型成效



涌现出一大批先进个人和团队

推动了云原生最佳实践的广泛认可和研讨

最佳实践对外输出，提升行业知名度

资源利用率极大提升

81

[100%, 目标81]

成熟度

80 *50%
平台得分 [目标81]

81 *50%
业务得分 [目标80]

79 *70%
利用率得分 [目标81]

84 *30%
CA得分 [目标82]

86 *70%
利用率得分 [目标81]

71 *30%
HPA得分 [目标78]

Crane - 工具链打通

Cloud Resource Analytics and Economics

云财务管理的一站式解决方案

开源项目: <https://github.com/gocrane>



Crane 能力矩阵

成本可视化

成本优化

成本分析

资源回收利用

成本预测

弹性

成本分配

服务分级和混合部署

质量保证

- 多维的成本展示
- 精准的浪费识别
- 基于云原生成熟度模型的评估体系
- 可靠的弹性
- 分级业务的质量保证
- 智能弹性和成本优化

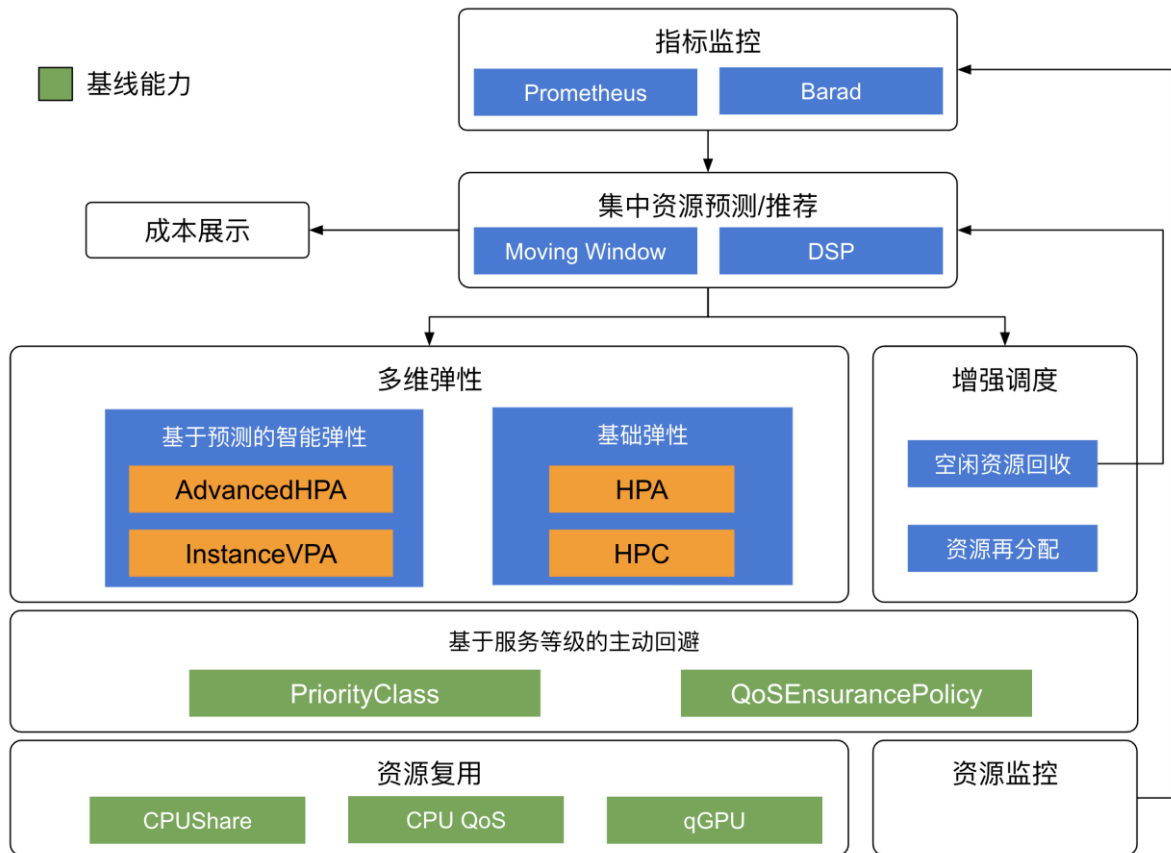
Crane 高层架构

Predictor 驱动整个生态系统

- 成本趋势展示
- 基于预测的资源回收与再分配
- 基于预测的智能弹性

基线能力

- 基于业务优先级的主动回避



资源预测

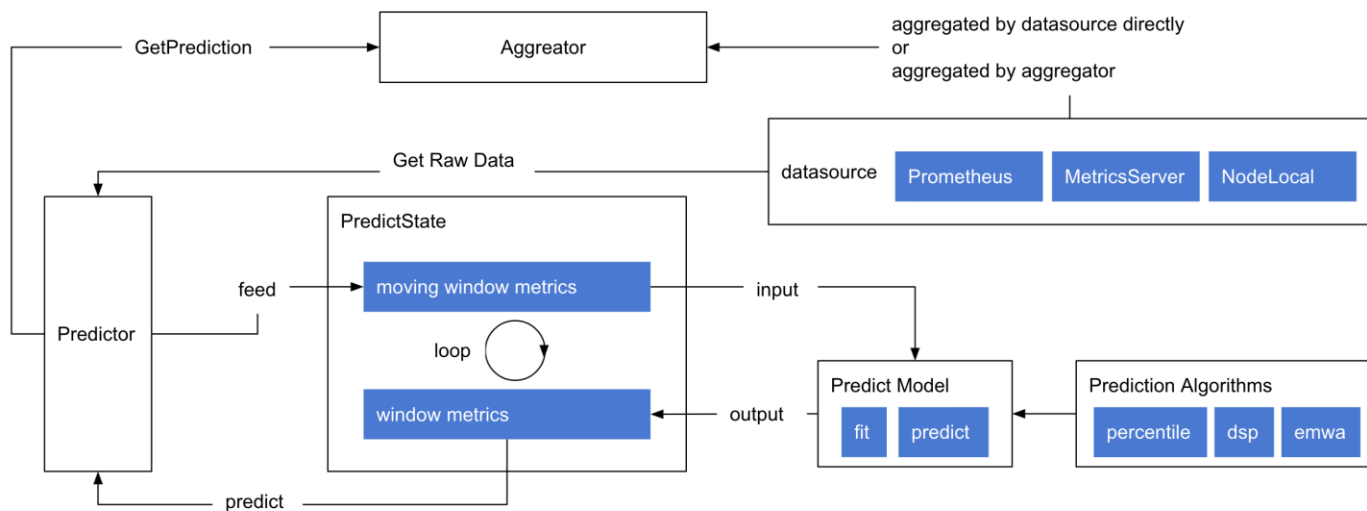
多数据源支持

可扩展的资源指标

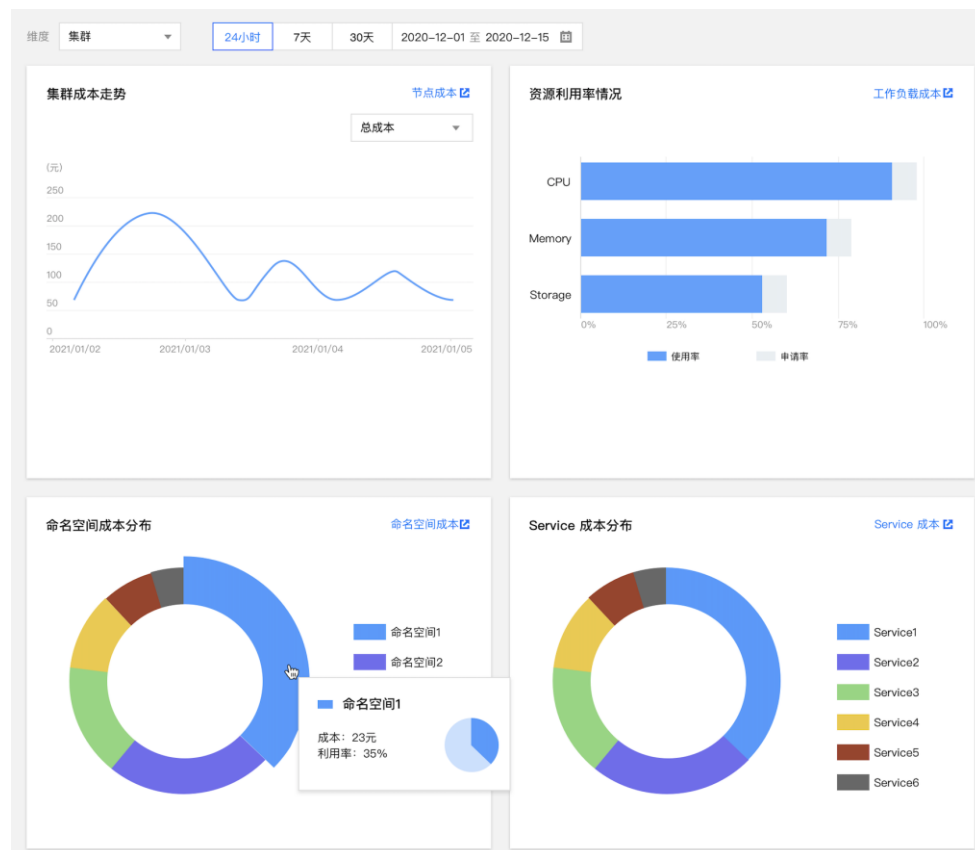
可自定义的算法

基于声明式 API 的预测

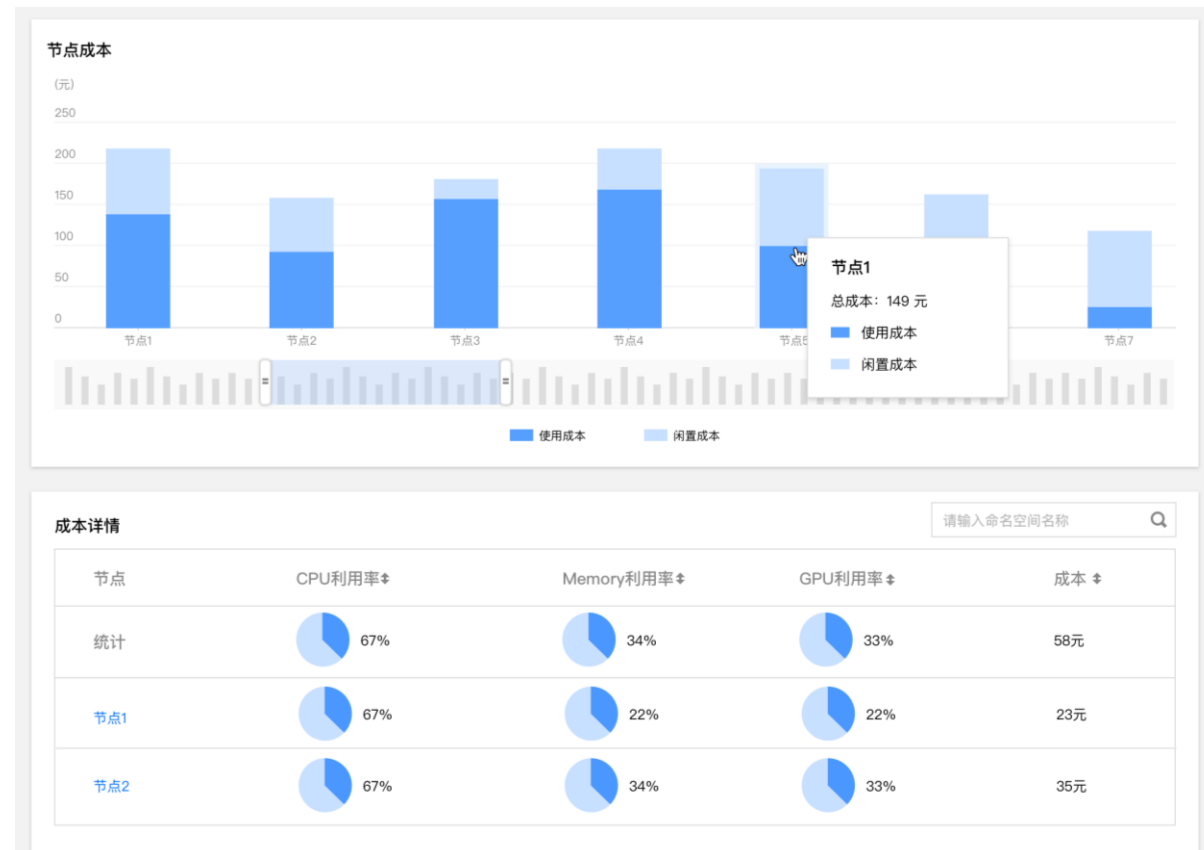
- TimeSeriesPrediction 对象定义资源预测规则
 - 预测结果按 PodGroup 汇聚，作为弹性能力的输入
 - 预测结果按 Node 汇聚，作为节点资源回收器的输入



多维的成本展示



- 基于预测结果的资源用量趋势分析



- 基于 FinOps 理念的成本展示

精准的浪费识别

闲置资源识别

资源浪费识别和基于预测结果的
Request 推荐

云原生成熟度模型的产品化支持

修改deployment : csi-cbs-controller中的Request

×

⚠ 注意

1. 当前 Request 设置过于不合理, 建议您更新以免资源浪费或供应不足
2. 将使用下列推荐的 Request 更新工作负载中容器的 Request
3. 若容器设置了 Limit, 新的 Request 与 Limit 会维持之前的比例
4. 更新 Request 后, 工作负载会滚动更新并重建 Pod, 更多请[查看](#)

容器名称	容器CPU Request	推荐CPU Request	容器Memory Request	推荐Memory Request
csi-provisioner	0.1核	0.011核	50Mi	17Mi
csi-attacher	0.1核	0.011核	50Mi	17Mi
csi-snapshotter	0.1核	0.011核	50Mi	17Mi
snapshot-controller	0.1核	0.011核	50Mi	17Mi
csi-resizer	0.1核	0.011核	50Mi	17Mi
cbs-csi	0.1核	0.011核	50Mi	17Mi

更新

取消

常规优化手段



弹性

- 按作业负载动态调节实例副本数或单实例的资源上
 - 负载高峰扩容以保证业务服务等级
 - 负载低谷缩容以回收资源减少浪费



混部

- 在负载低谷运行离线作业以提升总体资源利用率
- TKE-Scheduler 专为离线场景优化：gang-scheduling、all-or-nothing, 10X 性能提升

社区原生弹性能力的不足

HPA

- 业务指标驱动，指标的滞后导致业务突发流量时**来不及弹**。

VPA

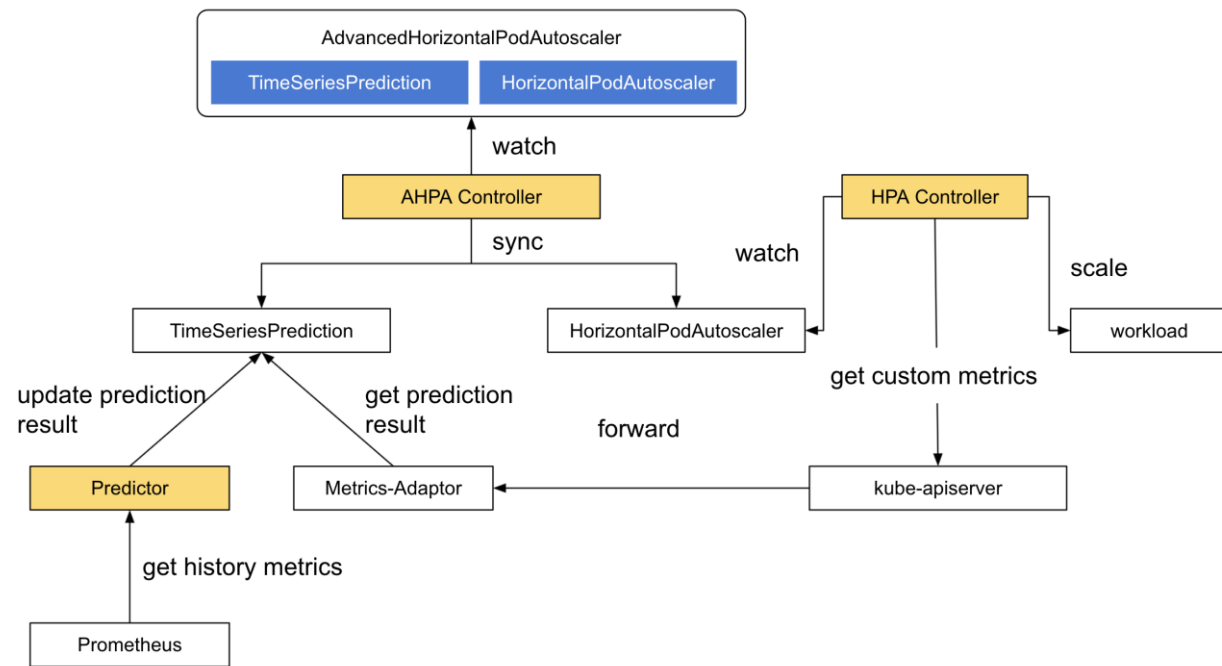
- 资源需求是 Pod 对象中的不可变属性，VPA 纵向调整资源后，Pod 需要重建，而现实场景中，**Pod 重建是大量业务不可接受的**。

CA

- 当集群资源不足时，CA 可按照既定规则扩容集群节点，但这对基础架构层面的**空闲资源和可靠性都有极高的要求**。

AHPA - 基于预测的横向伸缩

- 引入 `AdvancedHorizontalPodAutoscaler` 对象，该对象关联 `TimeSeriesPrediction` 和 HPA
- Metric-Adapter 通过 External Metric Provider 方式注册到 K8s 集群中，并将 `TimeSeriesPrediction.Status` 中的预测结果转化为 External Metrics
- HPA-Controller 通过 External Metric API 从 Metric-Adapter 获取应用的预测指标，该指标驱动 HPA 行为



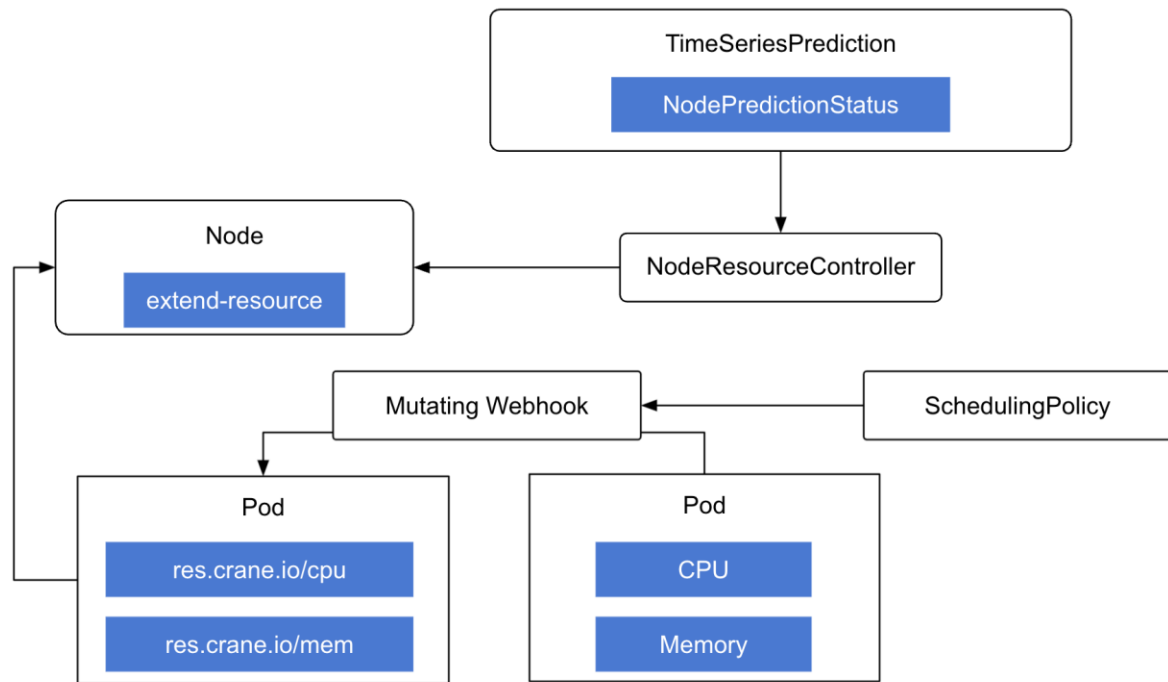
IVPA - 基于扩展资源回收的实时纵向伸缩

Craned 中的 **NodeResourceController** 控制器基于预测结果将空闲资源更新至节点 `extend-resource`。

引入 **SchedulingPolicy** 对象，为特定 `PriorityClass` 定义资源转化策略。

基于 **Mutating Webhook**，将特定 `PriorityClass` 的 Pod 中的资源请求转化为 `extend-resource`，实现回收资源再利用。

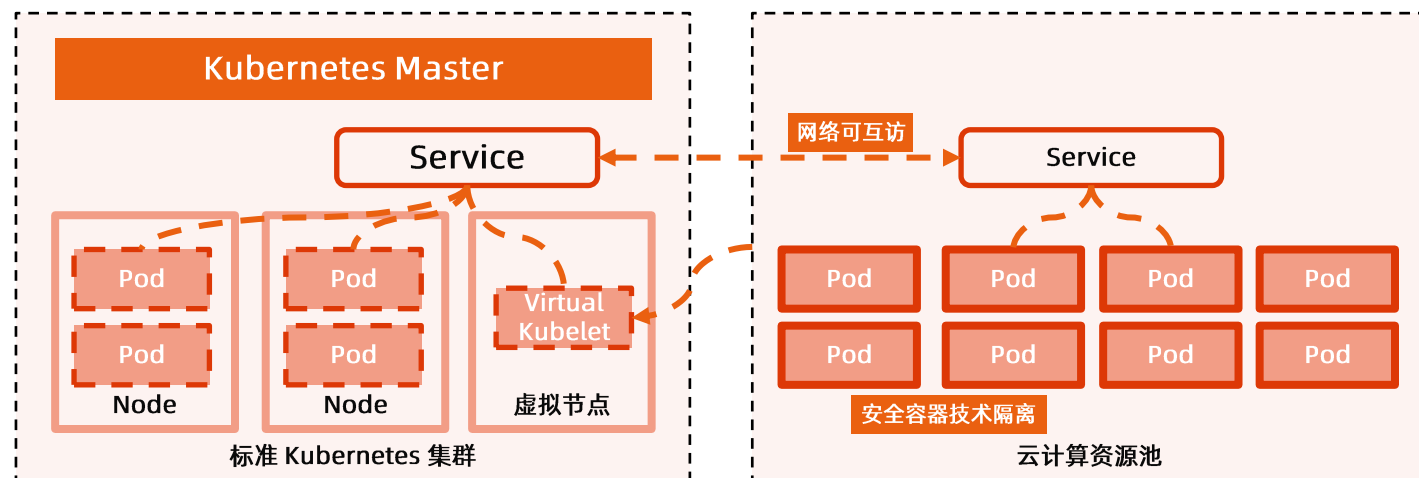
资源回收再利用可以被认为是广义的实时动态 VPA 能力（`InstanceVerticalPodAutoscaler`）。



基于分布式云虚拟节点技术的集群弹性

支持在标准 K8s 集群添加虚拟节点，虚拟节点打通弹性容器（EKS）的资源池，可将 Pod 调度为无服务器的弹性容器：

- 集群资源不足时，自动把 Pod 扩容到虚拟节点上；集群资源充足时，会优先缩容虚拟节点上的 Pod
- 每分钟支持快速扩容超过 1000 Pod
- 虚拟节点上的 Pod 与集群内所有资源（Service、Pod）网络互通
- 虚拟节点上的 Pod 兼容 Kubernetes 集群原生 Service 机制

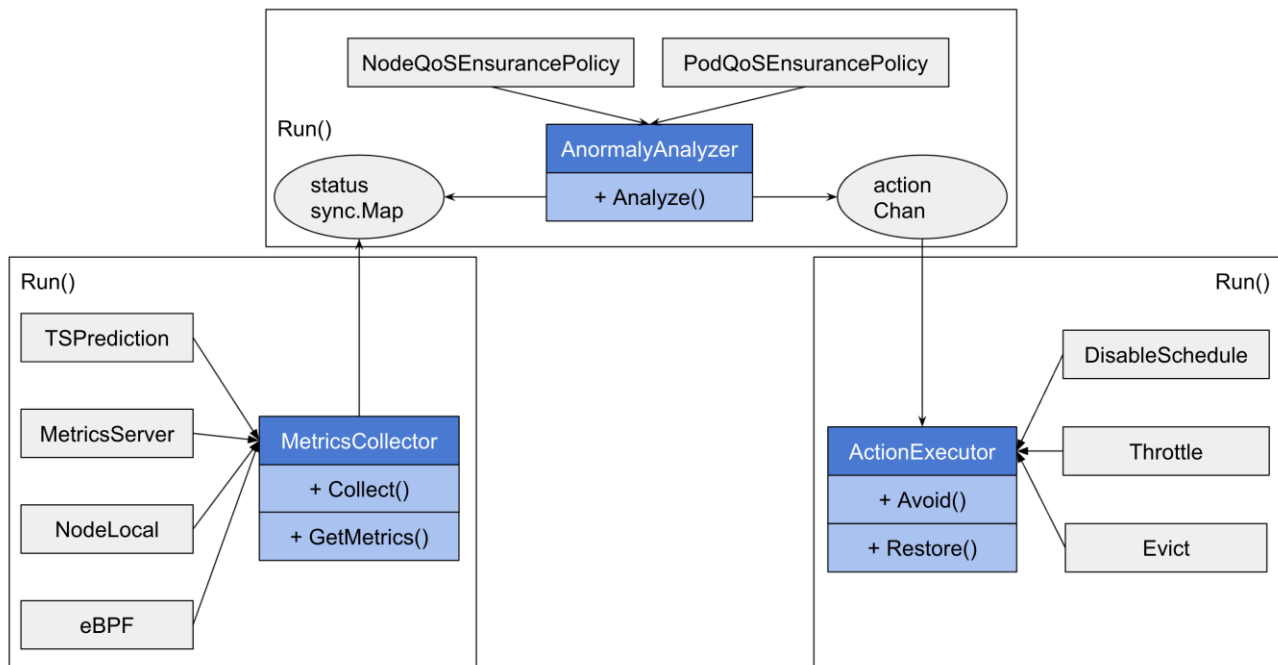


服务质量保证 - 安心拉升资源利用率的秘密

配合 **PriorityClass** 为业务定级。

针对节点水位和业务 SLO 进行
多维度异常指标检测。

当检测指标出现异常时，按预定义
优先级进行主动回避，确保高
优任务 SLO。



资源隔离 - OS 隔离 - RQSM 框架

支持开源 kernel 隔离 / QoS

- **CPU**: cgroup cpuset / cpu
- **HT**: thread siblings
- **Memory**: cgroup memory limit、reclaim
- **BLK**: cgroup DIO read / write、Buffered IO read、weight、xfs / ext4 disk quota
- **Network**: tc egress、优先级流控 TC HTB qdisc



THANKS

 极客时间 | 训练营